

UGANDA CHRISTIAN UNIVERSITY

A Centre of Excellence in the Heart of Africa

Faculty of Engineering, Design and Technology

Department of Computing and Technology

CSC3119: AI Model Deployment and Scalability

Track: Artificial Intelligence and Robotics

Predicting PM2.5 Air Quality Concentrations from AirQo Sensor Data

Assignment 4: Final Report

No.	Name	Registration No.
1	Charis Opol	S24B23/003
2	Cole Rodney Migisha	S24B23/006

October 2026

Abstract

We built a complete pipeline that predicts hourly PM2.5 concentration from AirQo low-cost sensor data: data preparation, exploratory analysis, model training, evaluation, explainability, an external test on a second dataset, August 2026 forecasts, monitoring with Evidently AI, and deployment as a web service on Render with a dashboard on Vercel. The supplied data contains 14,600 hourly records from 149 sensors, but readings exist on only eight days (1–3 January, 1–2 April and 1–3 June 2026). PM10 was removed from the pipeline: PM2.5 is the fine fraction of PM10, so the two rise and fall together because they are the same particles measured by the same instrument, not because PM10 drives PM2.5, and the literature points to meteorological conditions such as temperature and humidity as drivers of PM2.5 concentrations. PM2.5 is therefore predicted from temperature, humidity, time and sensor identity. Gaps are filled by windowed interpolation and time-of-day imputation, and a bidirectional LSTM imputer in the style of BRITS was added and tested. Extreme PM2.5 readings (1.5% of readings) are replaced by the nearest normal reading of the same sensor. SMOTE was tested on the training data and left out because it made every model less accurate. Thirteen models were compared under three validation setups. The best forecast model, a tuned SVR, reaches MAE 9.34 μg/m^3 (R^2 0.34) on the held-out June days. Adding the PM2.5 readings of the previous three hours to the tree models lowers this to 8.43 μg/m^3 (R^2 0.42) and makes the models far more stable, but such models need recent readings and cannot forecast weeks ahead. On an external dataset covering all of August for 125 of the sensors, the forecast models are no better than predicting the training mean, while the models with PM2.5 history keep part of their skill (MAE 13.7 μg/m^3). The bidirectional imputer is the most accurate imputer for gaps of six hours or more, but it does not improve the prediction models. With PM10 as an input the earlier version of the pipeline scored MAE 3.10 μg/m^3; that accuracy came from a co-measurement of the same particles, which is also unavailable ahead of time, not from what drives PM2.5.

Contents

1 Introduction **4**

1.1 What changed after the presentation 4

1.2 Project evolution and key decisions 5

2 Methodology overview **7**

3 Data **8**

4 Data preparation **9**

4.1 Approach and supporting literature 9

4.2 Cleaning 9

4.3 Extreme PM2.5 values 10

4.4 Hourly grid and offline time 10

4.5 Windowed interpolation 11

4.6 Time-of-day imputation 11

4.7 Bidirectional LSTM imputation (BRITS-style) 12

4.8 Calendars before and after cleaning 13

4.9 Features 14

5 Exploratory data analysis **14**

5.1 Sensor selection 14

5.2 Distributions 14

5.3 Calendar and time series 15

5.4 Correlation of temperature, humidity and PM2.5 16

6 Model implementation **17**

6.1 Set-up 17

6.2 Three validation setups 17

6.3 SMOTE: tested and left out 18

6.4 Models 18

6.5 Results 19

6.6 Predicted against actual 20

6.7 Data-rich against data-poor sensors 22

7 Evaluation of the imputation methods **23**

7.1 Masked-gap test 23

7.2 Effect on PM2.5 prediction 25

7.3 Does the bidirectional LSTM imputation add value? 26

7.4 Decision: which imputed data the final models are trained on 27

8 Model explainability with SHAP **28**

9 Sensor maps **30**

10 Second dataset test **32**

11 Forecasting and visualisation **34**

12 Model monitoring with Evidently AI **35**

13 Deployment **37**

13.1 Deployment history and choice of platforms 37

13.2 Local verification 38

13.3 Model service on Render 38

13.4 Dashboard on Vercel 39

13.5 Limitations of the deployment 41

14 Discussion and limitations **41**

14.1 Future development 42

15 Conclusion **43**

A Reproducing the results **45**

1 Introduction

AirQo, based at Makerere University, operates a network of locally manufactured low-cost air-quality monitors across African cities. Fine particulate matter (PM2.5) is the pollutant of greatest health concern: the particles penetrate deep into the lungs and bloodstream, and a four-year reference-grade study cited in the assignment brief found Kampala's annual average to be about $39\,\mu g/m^3$, roughly eight times the WHO annual guideline of $5\,\mu g/m^3$ [8]. Forecasting PM2.5 supports early warnings for the public and health services, and planning by NEMA and the ministries responsible for energy and transport.

The assignment asks for a deployable pipeline that goes from raw sensor data to a trained, evaluated, monitored and cloud-deployed model. This report documents each stage, the decisions taken and the evidence behind them. It describes the revised pipeline that followed the feedback on our presentation. The main changes are listed in Section 1.1. The AQI categories used throughout are those of Section 4 of the brief (Table 1).

Table 1: PM2.5 AQI categories used throughout the project, as implemented in the notebook.

PM2.5 ($\mu g/m^3$)	Category
0 – 9	Good
9.01 – 35.49	Moderate
35.5 – 55.49	Unhealthy for Sensitive Groups (USG)
55.5 – 125.49	Unhealthy
125.5 – 225.49	Very Unhealthy
above 225.49	Hazardous

1.1 What changed after the presentation

- **PM10 removed** from every table, plot, feature set and model, because it is a co-measurement of PM2.5 rather than a driver of it (Section 1.2).
- **Sensors profiled by offline time** and classed as data-rich or data-poor.
- **Gap filling in two steps:** windowed interpolation, then time-of-day imputation; plus a **bidirectional LSTM imputer** (BRITS-style) proposed by one of us.
- **Extreme PM2.5 values** replaced by the nearest normal reading of the same sensor.
- **SMOTE** tested on the training split, documented, and left out of the final models.
- **New models:** LightGBM, an LSTM, and tree models that also use the PM2.5 readings of the previous hours.
- **Three validation setups** instead of one.
- **New evaluation:** masked-gap imputation test, SHAP explanations, sensor maps, and an external test on a second dataset.
- **Plots revised:** calendars coloured by PM2.5 category, density plots instead of histograms, one combined time-series figure per site, and no AUC (it is not a regression metric).

1.2 Project evolution and key decisions

The project went through two pipelines. This subsection tells that story in order, including the ideas that were tried and dropped, because the dropped ideas explain the shape of the final pipeline. Table 2 summarises the decisions.

Phase 1: the first pipeline, with PM10. Zeros were converted to missing values, `device_name` was used as the sensor key, and coordinates were filled with the median of each site's own valid coordinates. Each sensor was reindexed to an hourly grid (758,112 sensor-hours) with an `is_offline` flag, and the predictors (temperature, humidity and PM10) were interpolated only across interior gaps of at most three hours; PM2.5 was never interpolated. The data were split chronologically (11,633 training rows up to 1 June, 2,642 test rows on 2–3 June), and the target was converted to a Z-score fitted on training rows only (mean 34.49, standard deviation 31.70 μg/m^3). Ridge, Linear, Polynomial, Random Forest, SVR and KNN models were tuned with `TimeSeriesSplit`. The selected tuned Random Forest reached MAE 3.10 μg/m^3, RMSE 7.00 μg/m^3 and R^2 0.898 on the test days, with 86.2% of hours in the correct AQI category (the untuned Random Forest: MAE 3.27 μg/m^3). PM10 carried about three quarters (76%) of that model's feature importance, while temperature and humidity carried about 1% each.

Why Phase 1 was abandoned. There are two reasons, one scientific and one practical.

- **PM10 is not a driver of PM2.5.** PM2.5 is the fine fraction of PM10: every particle counted in PM2.5 is also counted in PM10, and the two are measured by the same instrument in the same hour. They are strongly correlated because they share the same particles and the same sources, not because PM10 causes PM2.5. A model that uses PM10 learns this overlap, not how PM2.5 comes about. The literature on what drives PM2.5 concentrations points instead to emissions and to meteorological conditions: temperature and humidity affect how particles form, grow, disperse and are removed from the air [5, 6, 7] (Section 5.4). We therefore model PM2.5 from those conditions, together with time and sensor identity.
- **PM10 is not known in advance.** Because it is measured at the same moment as the target, a model that needs it cannot forecast. In the August forecast PM10 had to be replaced by each sensor's typical value for the hour, so the forecasts only repeated each site's usual daily pattern.

Most of Phase 1's accuracy came from this overlap, so the drop in accuracy after removing PM10 (Section 6.5) is expected. Following the feedback on our presentation, PM10 was removed from every stage.

Ideas tried and rejected, with the evidence.

- **KNN imputation of missing coordinates.** Six sites never report a valid coordinate. An early version borrowed coordinates from the five sites with the most similar temperature, humidity and pollution profile. This was rejected: a similar pollution profile does not mean a nearby location, so the six sites are left without coordinates and off the maps.
- **KNN filling of long gaps.** An early version filled every gap up to the last observation with a KNN imputer. For the five EDA sites 96–98% of each hourly series became imputed, and only about 12% of the values were distinct, because the imputer reused the same few

real readings for every hour with the same hour, weekday and month. The result was an artificial repeating sawtooth in the daily means. Long gaps are now left missing for PM2.5 and are filled for weather only, by the time-of-day method (Section 4.6).

- **Zero handling for weather dropouts.** In Phase 1, treating the 0 °C / 0% dropouts as missing, keeping them as 0, or adding a dropout flag changed the Random Forest's test MAE only between 3.24 and 3.30 $\mu g/m^3$, because PM10 dominated. The comparison no longer applies without PM10; the dropouts are now treated as missing and filled by the two-step gap filling.
- **Latitude and longitude as predictors.** They were removed: `device_name` already identifies the location, and six sensors have no coordinates.
- **SMOTE.** Tested in Phase 2 on the training split only. It made all eight models worse on the hold-out (Section 6.3), so `USE_SMOTE = False`; the experiment stays in the notebook.

Phase 2: the rebuilt pipeline. PM10 was dropped at loading. Extreme PM2.5 readings are replaced by the nearest normal reading of the same sensor (Section 4.3). Weather gaps are filled by windowed interpolation and then by a time-of-day profile with a local offset, both fitted on training rows only; a BRITS-style bidirectional LSTM imputer [9] was added and evaluated as a third method. Recent PM2.5 readings were added as features for the "nowcast" models, LightGBM and two LSTMs were added, and every model is scored under three validation setups.

The bidirectional LSTM imputer. It is the most accurate imputer on hidden values (21% lower error than the time-of-day method on average), but training the prediction models on its values changed their MAE by only −0.4% on average. The final models therefore keep the time-of-day imputation; the evidence is in Section 7.

Table 2: Decision log: the main decisions, the alternative considered, the evidence and the outcome.

Decision	Alternative	Evidence	Outcome
Remove PM10	Keep PM10 as an input (Phase 1)	PM2.5 is part of PM10, so PM10 is a co-measurement, not a driver; literature points to meteorological drivers; PM10 carried 76% of importance and is unknown when forecasting	PM10 dropped everywhere
Leave six sites without coordinates	KNN on pollution and weather profiles	Similar profiles do not imply nearby sites	Left missing
Leave long PM2.5 gaps missing	KNN filling of long gaps	96–98% of series imputed, ≈12% distinct values, artificial sawtooth	Rejected
Treat weather zeros as missing	Keep 0 or add a dropout flag	MAE 3.24–3.30 μg/m^3 in Phase 1, no real difference; zeros are sensor dropouts	Missing, then imputed
Drop latitude/longitude as predictors	Use coordinates	Device already identifies location; six sensors lack coordinates	Dropped
Replace extreme PM2.5	Keep or delete them	225 readings (1.5%) above the sensor's $Q_3 + 3\,\mathrm{IQR}$; maximum 998 μg/m^3	Nearest normal reading; originals kept
No SMOTE	SMOTE for regression, strengths 0.25 and 0.5	All eight models worse on the hold-out	`USE_SMOTE = False`
Time-of-day imputation for training	BRITS-style bidirectional LSTM	Better imputer (21%) but MAE change −0.4% over 15 cases	Time-of-day kept
Two saved models	One model	Forecast needs no recent readings; nowcast is more accurate	SVR (forecast), LightGBM + history (nowcast)
Render + Vercel deployment	RunPod; Hugging Face Spaces	RunPod credit too costly; server-side Spaces need a paid plan	Render service + Vercel dashboard

2 Methodology overview

Figure 1 shows the pipeline. All stages are implemented in a single notebook, `train.ipynb`, which runs unattended from start to finish. Everything that is fitted (hourly weather profiles, the bidirectional imputer, scalers, and SMOTE where it is used) is fitted on the training part of a split only.

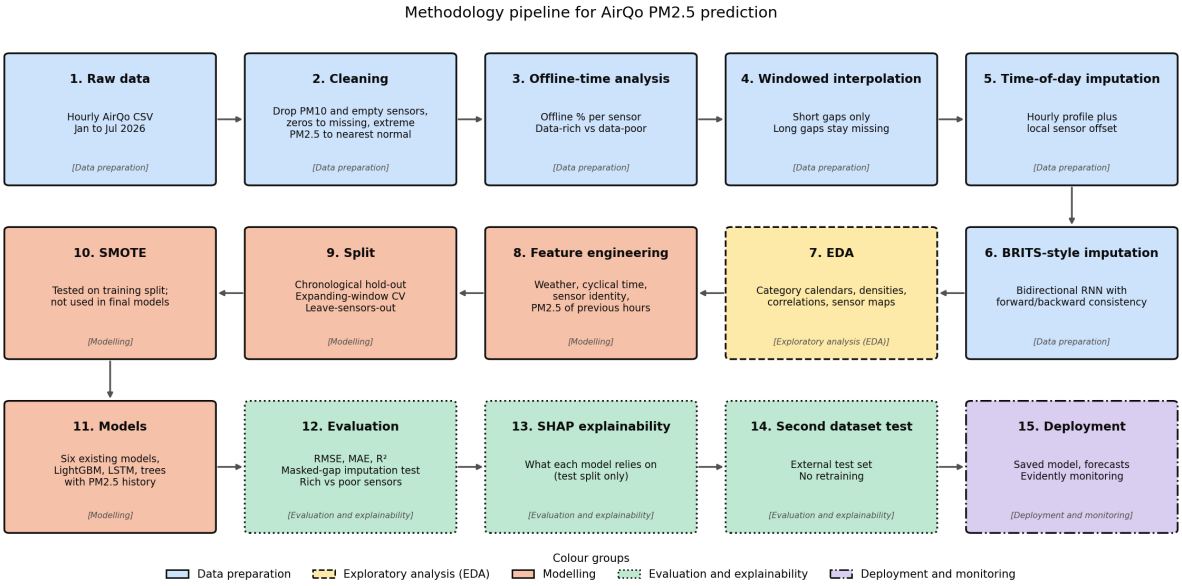


Figure 1: Methodology pipeline. Colours and border styles mark the five groups of stages. The bidirectional imputation step follows BRITS [9].

3 Data

The dataset (`Sensor Data Jan - July 2026.csv`) contains 14,600 hourly records with 12 fields: site name, timestamp, sample frequency, network (AirQo or AirGradient), latitude, longitude, temperature, humidity, calibrated PM2.5 and PM10, site ID and device name. It covers 149 devices. There are no duplicate rows and no duplicate device-hour pairs, so `device_name` together with the timestamp uniquely identifies an observation.

Three properties of the data shaped every later decision:

1. **Very sparse coverage in time.** Although the file is named January–July, readings exist on only eight days, in three short windows: 1–3 January, 1–2 April and 1–3 June 2026. The last reading is at 03:00 UTC on 3 June. That is 147 hours in which at least one sensor reported.
2. **Zero values that are not real readings.** The brief defines 0 or empty as missing. Table 3 gives the counts. About 28.8% of rows from AirQo devices report 0 °C and 0% humidity together, in long unbroken blocks, and no AirGradient row does: these are weather-sensor dropouts, not real readings.
3. **Uneven sensors.** Devices contribute between 20 and 147 records (median 105). The 149 devices sit at 145 named sites; some sites host more than one device, so the device, not the site, identifies a sensor.

Table 3: Missing and zero values in the raw file (14,600 rows). Zeros count as missing.

Column	Empty	Zero	Total missing
Latitude	1	3,085	3,086
Longitude	0	3,067	3,067
Temperature	54	4,066	4,120
Humidity	206	4,159	4,365
PM2.5	325	0	325
PM10 (dropped in Phase 2)	–		430

Gaps between readings. Of the gaps between consecutive readings of the same sensor, 13,668 are one hour long and 783 are longer, the longest about 150 days (between the bursts). **Sensor data quality.** Before PM10 was removed, PM2.5 exceeded PM10 in 22.4% of rows, which is physically impossible because PM2.5 is part of PM10, and 219 PM10 readings were below 1 μg/m^3. Thirteen PM2.5 readings are 200 μg/m^3 or more, and the largest, 998.4 μg/m^3 at Nansana East Ward (Wakiso), came with a PM10 of only 70.8 μg/m^3. These are signs of calibration and sensor errors in low-cost instruments, which is why extreme PM2.5 values are treated separately (Section 4.3).

AQI categories. Of the 14,275 raw PM2.5 readings, 1,249 are Good, 8,494 Moderate, 1,820 Unhealthy for Sensitive Groups, 2,565 Unhealthy, 137 Very Unhealthy and 10 Hazardous. After the extreme-value replacement the counts are 1,270, 8,627, 1,820, 2,446, 107 and 5.

A second file, `AirQo_August_2026_hourly_matched.csv`, became available later and is used only as an external test set (Section 10).

4 Data preparation

4.1 Approach and supporting literature

The preparation follows the general workflow of Febrian and Permai [1]: standardise null values, quantify missingness, explore distributions and outliers, analyse correlations, impute where justified, normalise, and split chronologically. AirQo's own studies describe the characteristics we found in this data: intermittent device availability, varying data completeness, and calibrated values from low-cost sensors [2, 3]. PM2.5 *targets* are never imputed for training or scoring, because that would create artificial labels.

4.2 Cleaning

- **PM10** is dropped immediately after loading, because it is a co-measurement of the same particles rather than a driver of PM2.5 (Section 1.2).
- **Zeros** in temperature, humidity, PM2.5, latitude and longitude are converted to missing. The zero temperature and humidity readings (about 4,100 rows) are a weather-sensor dropout: every zero temperature coincides with a zero humidity, they come in long unbroken blocks, and they occur only on AirQo devices.
- **Types:** timestamps are parsed as UTC, and `device_name` is used as the sensor identifier

because `site_id` is missing in about half the rows.

- **Empty sensors:** a sensor is removed if it has no valid reading at all or no valid PM2.5 reading. None met this rule, so none was dropped. Nine sensors report PM2.5 but never report weather; they were kept and their weather is imputed.
- **Coordinates** are filled with the median of the site's own valid coordinates. Six sensors have no valid coordinate and are left without one.

4.3 Extreme PM2.5 values

A few PM2.5 readings are far outside anything else the same sensor reports; the largest is 998 μg/m^3. A reading is treated as extreme when it is above the sensor's own far-out limit, $Q_3 + 3\,\mathrm{IQR}$. Each extreme reading is replaced by the nearest reading *in time* from the *same sensor* that is within the normal range, looking at most six hours away; if none exists, the sensor's median normal reading is used.

This flagged 225 of 14,275 readings (1.5%) at 59 sensors. 223 were replaced by a normal reading at most five hours away (median one hour) and two by the sensor median. The mean of PM2.5 moves from 32.7 to 31.6 μg/m^3, the standard deviation from 30.4 to 26.8 μg/m^3, and the maximum from 998 to 415 μg/m^3 (Figure 2). The original values are kept, and Section 6.5 also reports the models' error against the untouched readings.

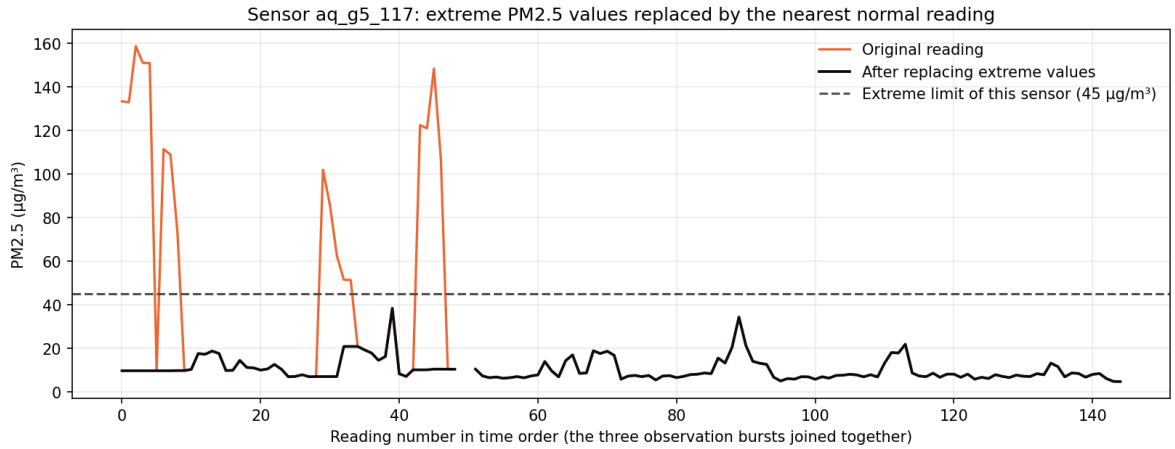


Figure 2: The sensor with the most extreme readings, before and after replacement by the nearest normal reading.

4.4 Hourly grid and offline time

Each sensor was reindexed to a complete hourly grid from 1 January to 31 July 2026 (5,088 hours; 758,112 sensor-hours). A sensor is *offline* in an hour when it has no valid PM2.5 reading. Over the whole study period every sensor is between 97.1% and 99.6% offline, because the file only contains eight days, so that figure cannot separate sensors. The offline percentage over the 147 network-active hours can, and is used to classify them (Table 4, Figure 3).

Table 4: Sensor classes by offline time over the network-active hours.

Class	Rule (offline %)	Sensors	Mean offline %
Data-rich	at most 20	61	8.6
Intermediate	between 20 and 50	40	34.7
Data-poor	at least 50	48	68.3

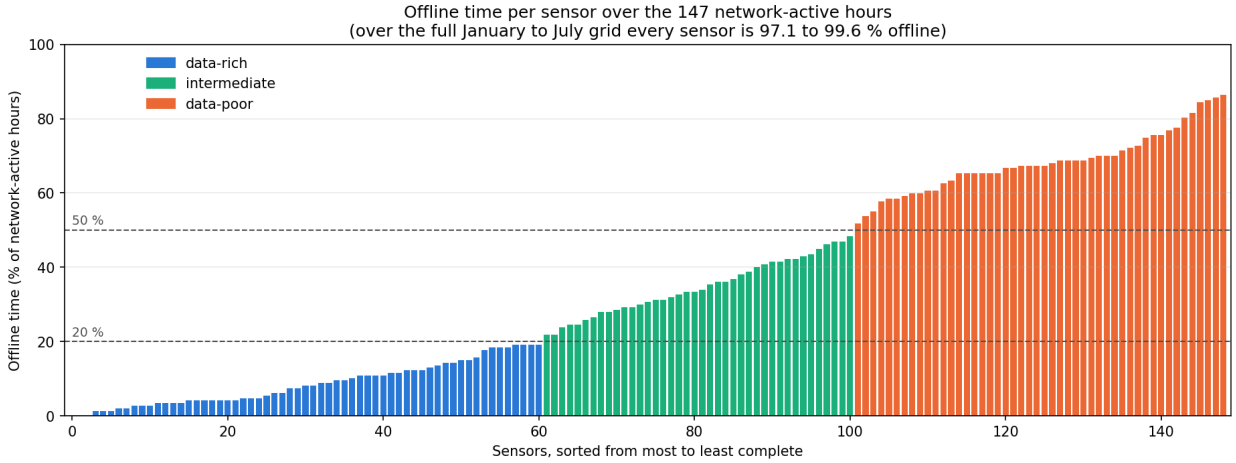


Figure 3: Offline time of each sensor over the network-active hours, sorted, with the two class limits.

4.5 Windowed interpolation

A missing hour is filled by linear interpolation only when the gap is at most three hours long, has valid readings on both sides, and the 24-hour window centred on it holds at least six real readings. It runs separately for each sensor, and the window, gap limit and minimum count are parameters. Longer gaps stay missing. It filled 555 temperature and 551 humidity values.

4.6 Time-of-day imputation

Longer gaps inside the observation windows are estimated from the daily cycle. The mean and variance of each variable are computed per hour of day for each sensor, falling back to the sensor's network group and then to all sensors when a sensor has too few readings. The sensor's own readings within 12 hours of the gap give a local offset from that profile, and the missing value is the profile value plus the offset (Figure 4). For the models, the profiles are refitted on training rows only.

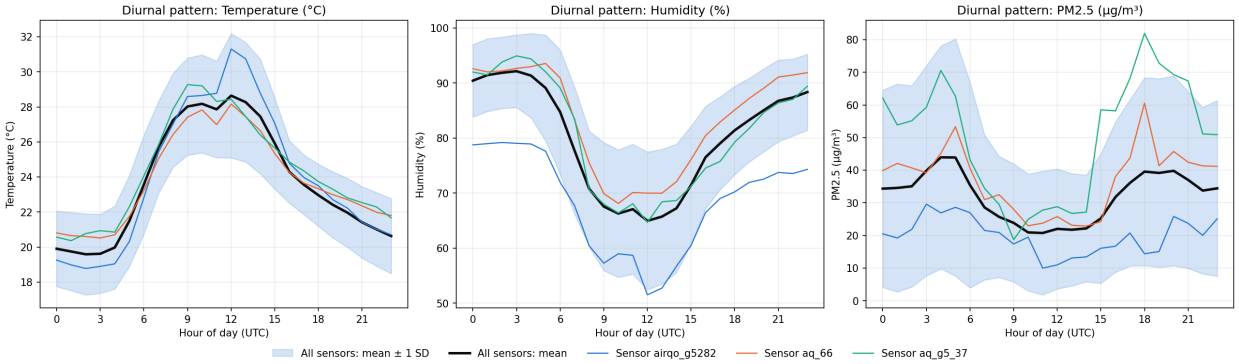


Figure 4: Hourly pattern of temperature, humidity and PM2.5: all-sensor mean with one standard deviation, and three individual sensors. Hours are UTC (local time is UTC+3).

Hiding 10% of the known values and imputing them gives the errors in Table 5. For six-hour gaps, which interpolation leaves empty, the method is better than the hourly profile alone and well below filling with the sensor mean. Figure 5 compares the distributions before and after gap filling.

Table 5: Masked-value validation of the time-of-day imputation (mean absolute error).

Variable	Gap	Profile + offset	Profile only	Sensor mean	Interpolation
Temperature (°C)	1 h	1.40	1.72	3.01	0.61
Temperature (°C)	6 h	1.53	1.66	3.14	not filled
Humidity (%)	1 h	5.33	6.74	10.20	2.06
Humidity (%)	6 h	6.16	6.43	10.82	not filled
PM2.5 (μg/m^3)	1 h	11.84	18.61	17.46	7.48
PM2.5 (μg/m^3)	6 h	12.54	19.04	17.95	not filled

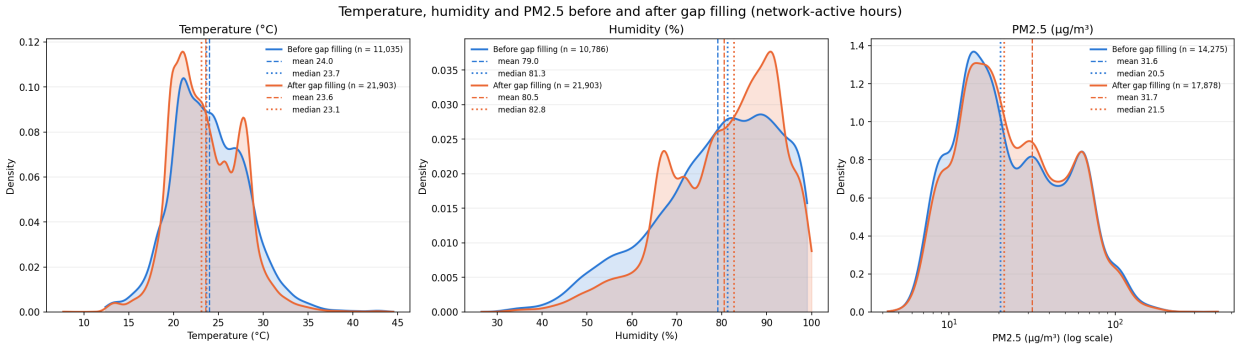


Figure 5: Density of temperature, humidity and PM2.5 before and after gap filling, with means (dashed) and medians (dotted).

4.7 Bidirectional LSTM imputation (BRITS-style)

A third imputation method implements an idea proposed by one of us: a recurrent model reads each sensor series forwards and backwards, each direction estimates every value before it sees it, and a consistency loss makes the two directions agree. This is supervised learning with an added consistency loss. It follows BRITS [9]: an LSTM per direction whose hidden state decays with the time since the last reading; a history-based estimate, a feature-based estimate from the other

variables of the same hour, and a learned blend; a reconstruction loss on observed values only; and the forward/backward consistency loss.

Our version differs from the paper in the data (three variables from sensors that report in three short bursts), in adding the hour of day and the sensor's offline share as always-known conditioning inputs, in modelling PM2.5 on a log scale, in hiding blocks of 1 to 12 hours during training, and in having no classification head. It is written in TensorFlow; the PyPOTS library was not used because it does not report the consistency term separately or accept conditioning inputs. Scalers and the model are fitted on the training period only and each split is imputed separately. Figure 6 shows the training curves. Its accuracy and its effect on the models are evaluated in Section 7.

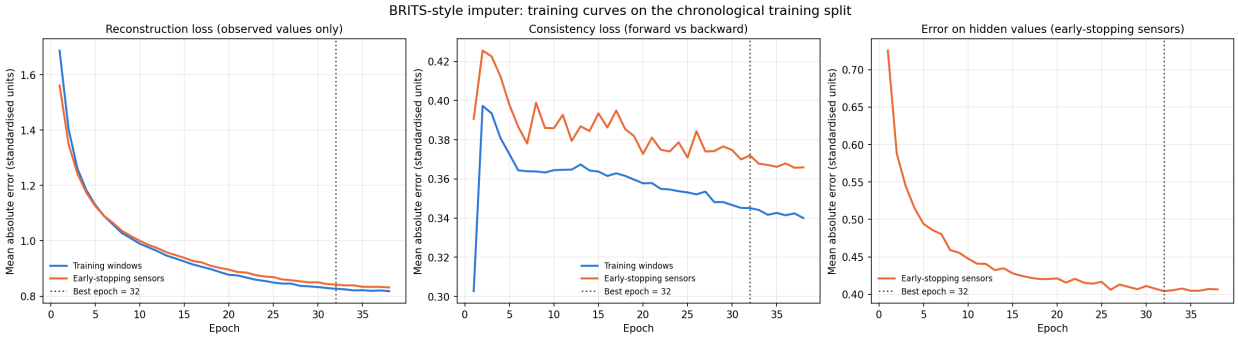


Figure 6: Training curves of the bidirectional imputer: reconstruction loss, consistency loss, and the error on deliberately hidden values used for early stopping.

4.8 Calendars before and after cleaning

Figure 7 shows the daily network mean before and after cleaning, coloured by PM2.5 category. Cleaning fills hours inside the observed days; it does not create new days, because long gaps are left missing on purpose. Coverage on the six full observation days rises from about 58–70% of sensor-hours to about 75–86%.

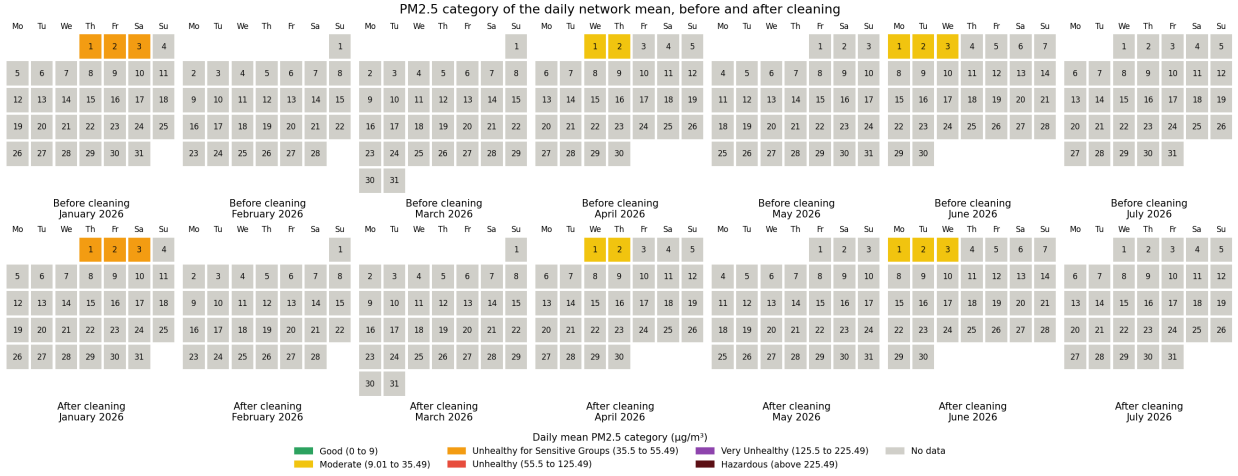


Figure 7: PM2.5 category of the daily network mean, before (top) and after (bottom) cleaning. Grey days have no data.

4.9 Features

- **Weather:** temperature and humidity.
- **Time:** hour of day, day of week and month, each as a sine/cosine pair.
- **Sensor identity:** `device_name`, one-hot encoded.
- **PM2.5 history** (for the models that use it): the readings of the same sensor 1, 2 and 3 hours earlier, their mean, and how many of them exist. Only real earlier readings are used, so the value being predicted cannot leak into its own inputs.
- **Not used:** PM10, coordinates (the device already identifies the location) and administrative fields.

5 Exploratory data analysis

5.1 Sensor selection

EDA focuses on five sensors inside Kampala District, as the brief requires. Single-device Kampala sites were ranked by the number of hours with PM2.5, temperature and humidity all present, and a site is only eligible if it has readings in every month that contains data, so that no EDA panel is empty (Table 6).

Table 6: The five Kampala sensors used for EDA.

Site	Device	PM2.5 records	Complete records
Nakasero II, Kampala	`aq_66`	147	147
Lower Nsooba, Kawempe	`aq_g507`	144	144
Salama Road	`ag_588c81267534`	142	142
Namungona Primary School	`airqo_g5365`	135	135
Kabusu, Natete Road	`aq_g5_100`	134	134

5.2 Distributions

Distributions are shown as kernel-density curves, one panel per variable, with the mean and median marked (Figures 8 and 9). PM2.5 is strongly right-skewed, so its panel uses a log axis. Data-poor sensors record somewhat higher PM2.5 than data-rich ones.

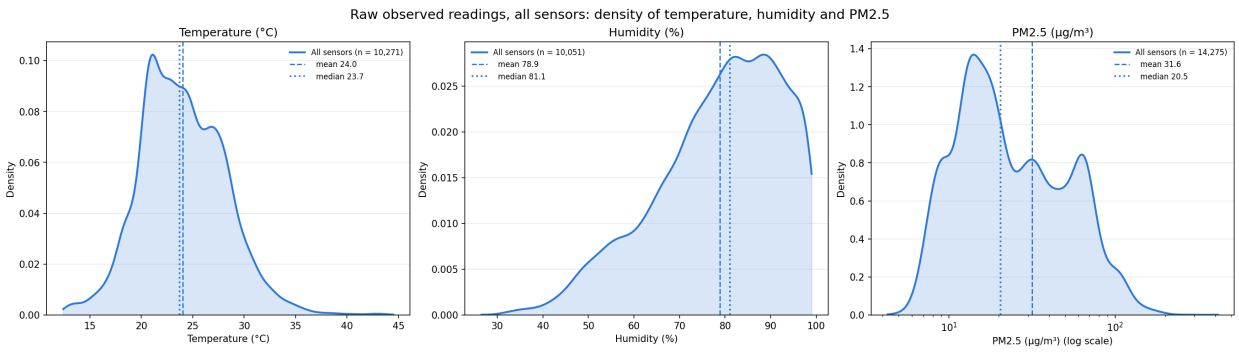


Figure 8: Density of observed temperature, humidity and PM2.5, all sensors.

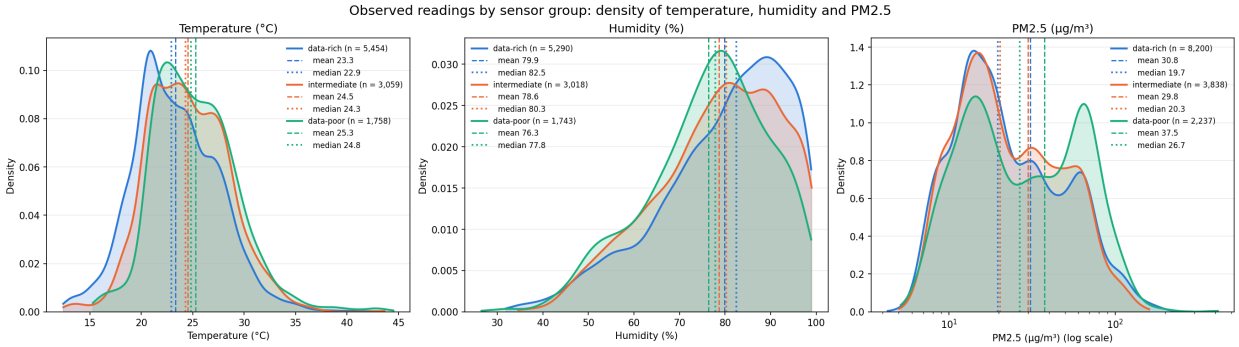


Figure 9: The same densities by sensor group: data-rich, intermediate and data-poor.

5.3 Calendar and time series

Figure 10 shows the calendar for the five Kampala sites. Each site also has one combined figure with temperature, humidity and PM2.5 on a shared time axis and three colour-matched y-axes; Figure 11 shows the network mean. Because the data come in three bursts months apart, the time axis is cut into the three observation windows.

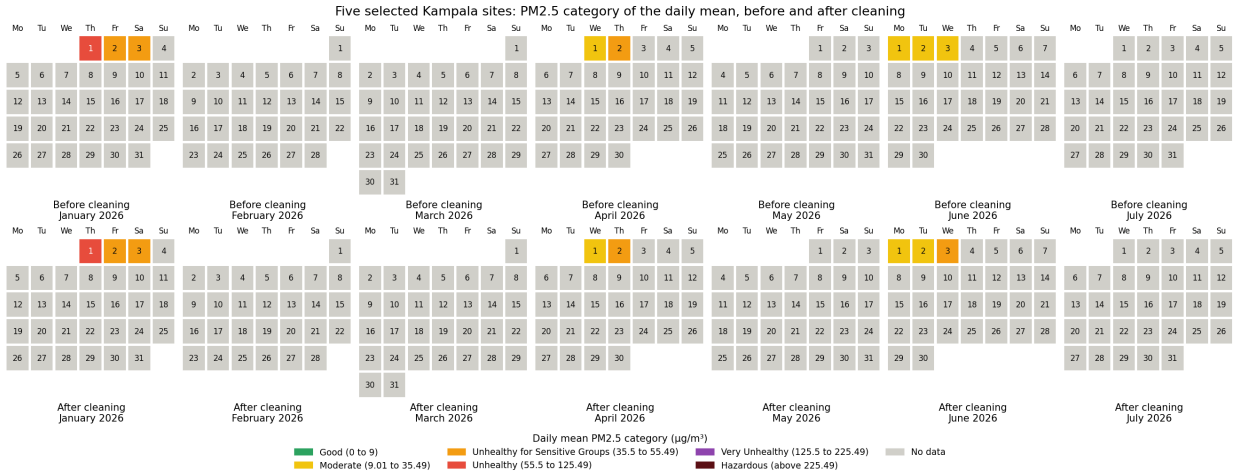


Figure 10: Five Kampala sites: PM2.5 category of the daily mean, before and after cleaning.

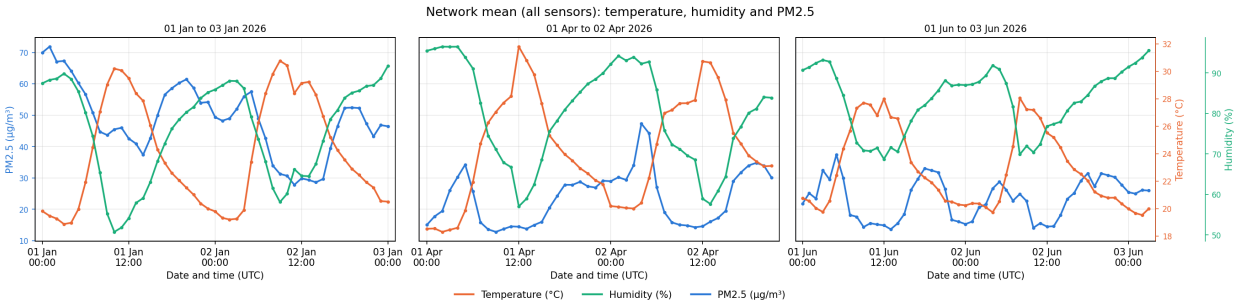


Figure 11: Network mean of temperature, humidity and PM2.5 on a shared time axis. PM2.5 peaks in the early morning and evening.

5.4 Correlation of temperature, humidity and PM2.5

Table 7 gives the Pearson and Spearman correlations over all sensors. Temperature and humidity are strongly and inversely related; their direct link with PM2.5 is weak. Within the five Kampala EDA sites the links are stronger and consistent in sign: PM2.5 is negatively correlated with temperature (r from -0.13 at Nakasero to -0.51 at Salama Road) and positively with humidity (r from 0.08 at Namungona to 0.45 at Kabusu).

These signs agree with the literature on how weather affects PM2.5. Warmer hours usually come with a deeper, better-mixed boundary layer that dilutes particles, while cool, humid hours at night and early morning trap them near the ground; Wang and Ogawa [5] found PM2.5 negatively correlated with temperature in Nagasaki. Warmth can also speed up photochemical formation of secondary particles, which pushes the other way, so the net temperature effect depends on place and season; Yang et al. [6] found it negative in autumn but positive in winter across Chinese cities. Humidity has two opposing effects: moist air lets hygroscopic particles take up water and grow, and favours secondary aerosol formation, which raises PM2.5, while very humid conditions with rain remove particles by wet deposition. Which effect wins depends on particle composition and climate: Yang et al. [6] found the humidity–PM2.5 correlation positive in North China but negative in most other regions, with the positive link strongest in winter and spring, and Wang and Ogawa [5] found the sign of the humidity relationship changing at a threshold (see also [7]). With only eight days of data we cannot separate these mechanisms; the correlations are consistent with them but do not prove them.

Table 7: Correlation of PM2.5, temperature and humidity (all sensors, observed readings).

Pair	Pearson	Spearman
PM2.5 and temperature	-0.14	-0.15
PM2.5 and humidity	0.09	0.10
Temperature and humidity	-0.81	-0.81

Figure 12 shows the lagged cross-correlations within sensors, up to 24 hours each way. A positive lag means the first variable of the pair comes first. Temperature and humidity peak at lag 0 ($r = -0.88$). Temperature and PM2.5 are weakly negative at lag 0 ($r = -0.24$) and at one hour (-0.25); the largest value, -0.30 at 24 hours, is the daily cycle repeating. Humidity and PM2.5 are weakly positive at lag 0 (0.18) and reach -0.31 at nine hours, again the daily cycle seen half a day apart. Same-hour and one-hour-earlier weather are therefore the only weather lags worth using; longer lags repeat what the hour-of-day features already carry. These are correlations, not evidence that weather causes PM2.5 to change.

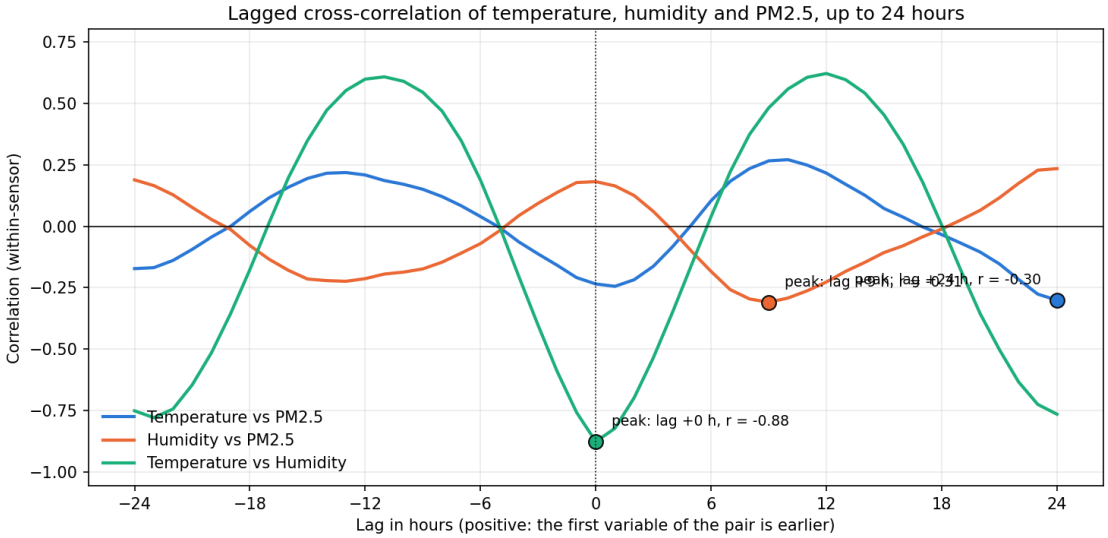


Figure 12: Lagged cross-correlation of the three pairs, with the peak of each marked.

6 Model implementation

6.1 Set-up

- **Rows:** the 14,275 hours with a PM2.5 reading.
- **Target:** the Z-score of PM2.5, fitted on training rows only; predictions are transformed back to $\mu g/m^3$.
- **Preprocessing**, inside each pipeline: numeric inputs standardised, a remaining missing value filled with the training median, `device_name` one-hot encoded with unseen devices ignored.
- **Tuning:** grid or randomised search with `TimeSeriesSplit` on the real training rows.
- **Metrics:** MAE (the selection metric), RMSE and R^2, plus AQI category accuracy. AUC is not used because it is not a regression metric.

6.2 Three validation setups

Table 8: The three validation setups.

Setup	Training and validation rows	Question
Chronological hold-out (main)	Train on 1 January – 1 June (11,633 rows); test on 2–3 June (2,642 rows)	How well does the model predict a later period?
Expanding-window CV	Validate on 1 April, 2 April and 1 June, training on everything earlier each time	Is that result stable across periods?
Leave-sensors-out	Three folds, each holding out about 50 sensors completely	Can the model predict a sensor it has never seen?

6.3 SMOTE: tested and left out

SMOTE [12] creates synthetic examples of rare classes by interpolating between neighbours. It was not used in the earlier version. Because PM2.5 is continuous, it was applied in the way proposed for regression [13]: training rows are grouped into PM2.5 classes, the rare classes are oversampled, and the target is interpolated with the features. An air-quality example of SMOTE for PM2.5 classes is [14]. Only training rows are ever resampled, and hyperparameters are tuned on real rows.

Table 9 shows the result on the untouched hold-out. SMOTE made all eight models worse at both strengths. Oversampling the rare high-pollution hours raises the average PM2.5 a model is trained on; the inputs cannot tell the model where those hours belong, so it raises its predictions everywhere, and the hold-out days are cleaner than the training period (mean 23 against 33 μg/m^3). Cross-validation inside the training rows preferred a strength of 0.25, but that gain did not carry over to the hold-out for any model. The final models are therefore trained on real rows only; the experiment remains in the notebook.

Table 9: Hold-out MAE (μg/m^3) without SMOTE and with SMOTE on the training split at two strengths.

Model	Without SMOTE	SMOTE 0.25	SMOTE 0.5
Random Forest (untuned)	9.61	10.10	10.60
Random Forest (tuned)	9.63	10.12	10.73
KNN	10.01	10.41	10.84
LightGBM	9.82	28.67	23.39
SVR	9.34	30.22	28.02
Ridge	22.00	35.11	38.43
Linear / Polynomial	23.86	33.86	36.47

6.4 Models

Thirteen models were compared:

- the six from the earlier version, trained with scikit-learn [16]: Linear, Polynomial and Ridge regression, Random Forest [15] (untuned and tuned), SVR and KNN;
- **LightGBM**, a gradient-boosted tree model, with a light random search;
- **two LSTMs** on six-hour sliding windows per sensor, one with weather and time only and one that also sees the PM2.5 reading of the previous hour at each step, with early stopping (Figure 13);
- **Random Forest and LightGBM with PM2.5 history**, the tree models with the history features of Section 4;
- a baseline that always predicts the training mean.

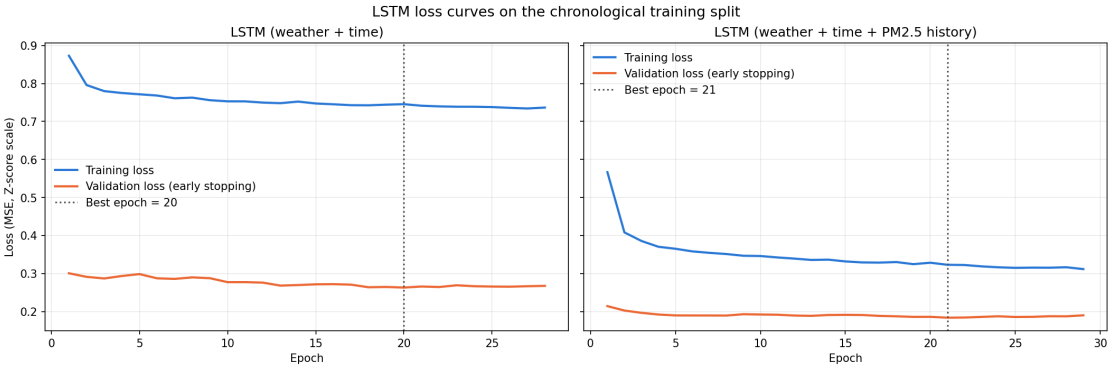


Figure 13: LSTM training and early-stopping validation loss.

Two kinds of model result. *Forecast models* predict from weather, time and sensor identity alone, so they can produce the August forecast. *Nowcast models* (those with PM2.5 history) predict the current hour from the sensor's last few hours; they are more accurate but need recent readings.

6.5 Results

Table 10: Results under the three validation setups (mean over folds, $\mu g/m^3$), sorted by hold-out MAE.

	Hold-out			CV	Unseen sensors	
Model	MAE	RMSE	R^2	MAE	MAE	R^2
LightGBM + PM2.5 history	**8.43**	14.21	0.42	12.08	9.49	0.63
Random Forest + PM2.5 history	8.50	14.25	0.41	10.25	9.03	0.65
SVR (tuned)	**9.34**	15.14	0.34	14.68	15.09	0.27
Random Forest (untuned)	9.61	16.14	0.25	27.02	14.90	0.22
Random Forest (tuned)	9.63	16.17	0.24	26.82	14.92	0.22
LightGBM (tuned)	9.82	16.08	0.25	27.45	15.13	0.27
KNN (tuned)	10.01	16.05	0.26	16.69	15.60	0.24
LSTM with PM2.5 history	10.89	15.58	0.30	12.18	9.41	0.65
LSTM (weather and time)	12.22	18.53	0.01	20.70	15.61	0.26
Baseline (training mean)	17.60	21.17	−0.30	24.11	20.33	−0.03
Ridge (tuned)	22.00	25.37	−0.86	23.87	16.42	0.22
Linear (tuned)	23.86	27.87	−1.25	24.73	16.37	0.21
Polynomial (tuned)	23.86	27.87	−1.25	24.72	16.67	0.21

Which model is best. The tree models with PM2.5 history are the best overall (Table 10). LightGBM with history has the lowest hold-out MAE ($8.43\,\mu g/m^3$) and the Random Forest with history is the most consistent across the three setups (8.5, 10.3 and $9.0\,\mu g/m^3$). The history features lower the hold-out MAE of the Random Forest by 11.6% and of LightGBM by 14.2%, and the gain is much larger in the other two setups, where the same models without history collapse (about $27\,\mu g/m^3$ in the expanding-window CV).

This agrees with Febrian and Permai [1], who found for Jakarta that historical pollutant readings alone were sufficient for short-term PM2.5 forecasting and that meteorological inputs added little. The leave-sensors-out setup plays the same role as the station-blinded validation of Gürtepe et al. [4]: it measures how well a model predicts at a location it has never seen, and, as in their study (RMSE 9.21 against $8.28\,\mu g/m^3$ for their temporal evaluation), it scores worse than our

chronological hold-out.

Among the forecast models the tuned SVR is best (MAE 9.34 μg/m^3, R^2 0.34). It is saved as `best_pm25_model.joblib` and produces the August forecast; LightGBM with history is saved separately as `best_pm25_nowcast_model.joblib`. The forecast models beat the training-mean baseline clearly on the hold-out but not in the expanding-window CV, where the tree models are worse than the baseline: trained on one or two bursts, they learn that month's level and carry it into the next.

Against the untouched readings. The scores above compare predictions with PM2.5 in which extreme values were replaced (53 of the 2,642 hold-out readings). Against the original readings the MAEs are 9.79 μg/m^3 for LightGBM with history and 10.62 μg/m^3 for the SVR.

AQI categories. The SVR puts 75.2% of hold-out hours in the correct category and LightGBM with history 76.9%; always predicting the training mean scores 74.4%, because most hours are Moderate. Category accuracy is a weak yardstick on this data.

6.6 Predicted against actual

Figures 14 to 16 compare predictions with the actual readings on the hold-out. The models follow the level of a sensor but not its sharp peaks. The mean residual is close to zero for the SVR ($-1.0\,\mu$g/m^3) and for LightGBM with history ($-0.5\,\mu$g/m^3), and positive for the LSTM with history ($+4.4\,\mu$g/m^3).

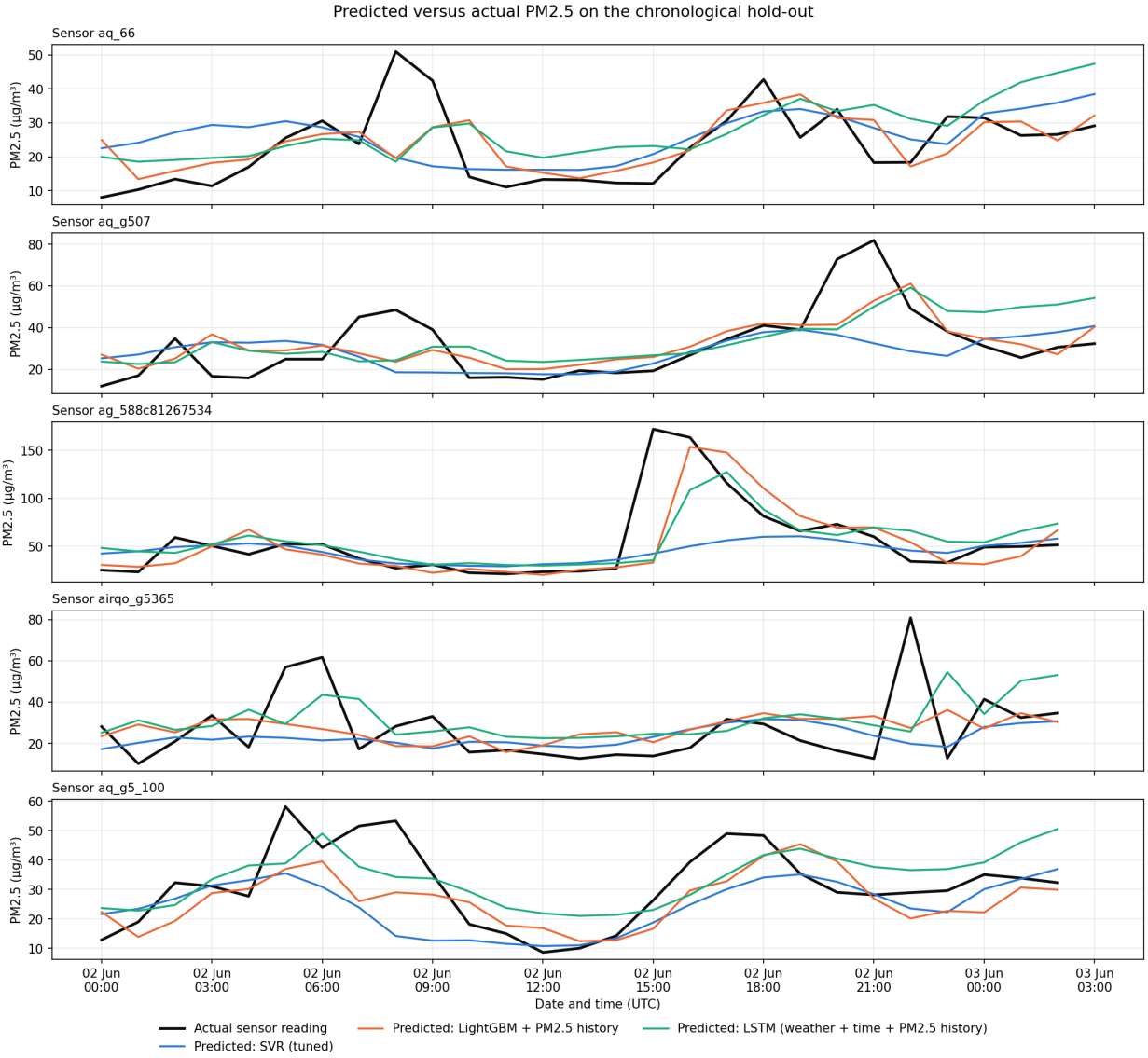


Figure 14: Predicted and actual PM2.5 for the five Kampala sensors on the hold-out.

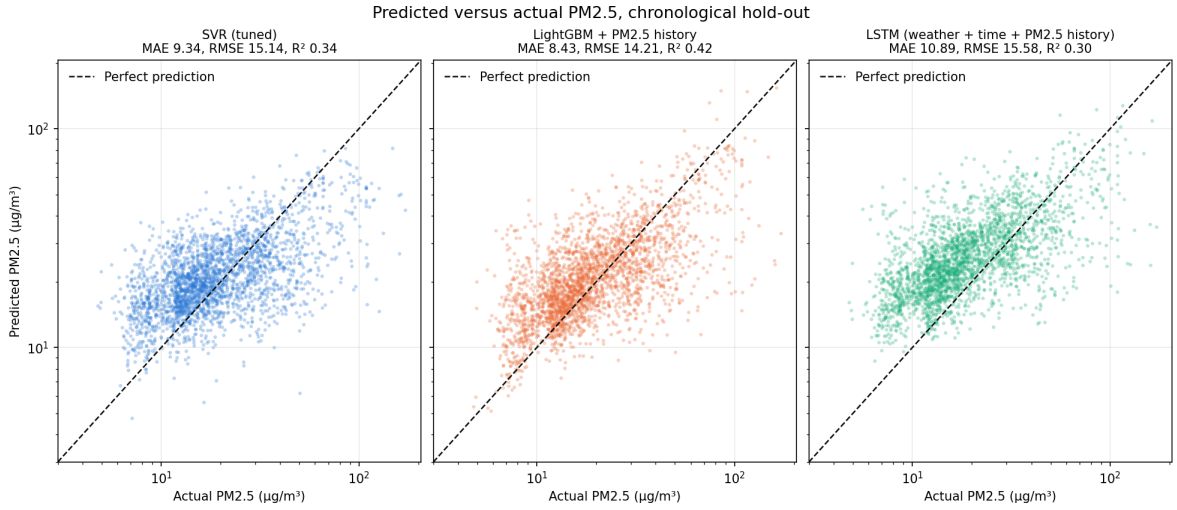


Figure 15: Predicted against actual PM2.5 (log axes); the dashed line is a perfect prediction.

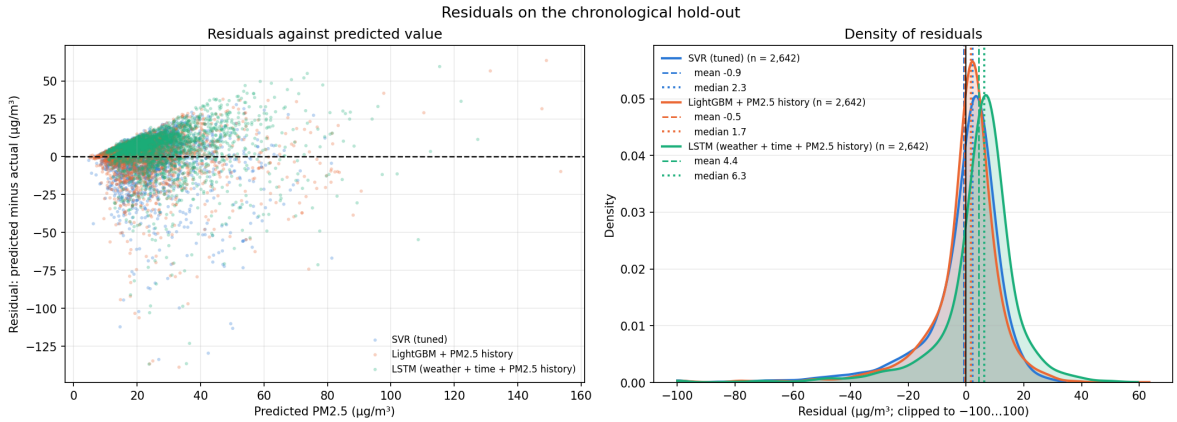


Figure 16: Residuals against the predicted value, and the density of the residuals.

Figure 17 shows the actual and predicted daily network mean as calendars in the same PM2.5 category colours. The category matches on all five out-of-sample days, but hour by hour only 71% of sensor-hours fall in the right category: averaging over sensors and hours hides the peaks.

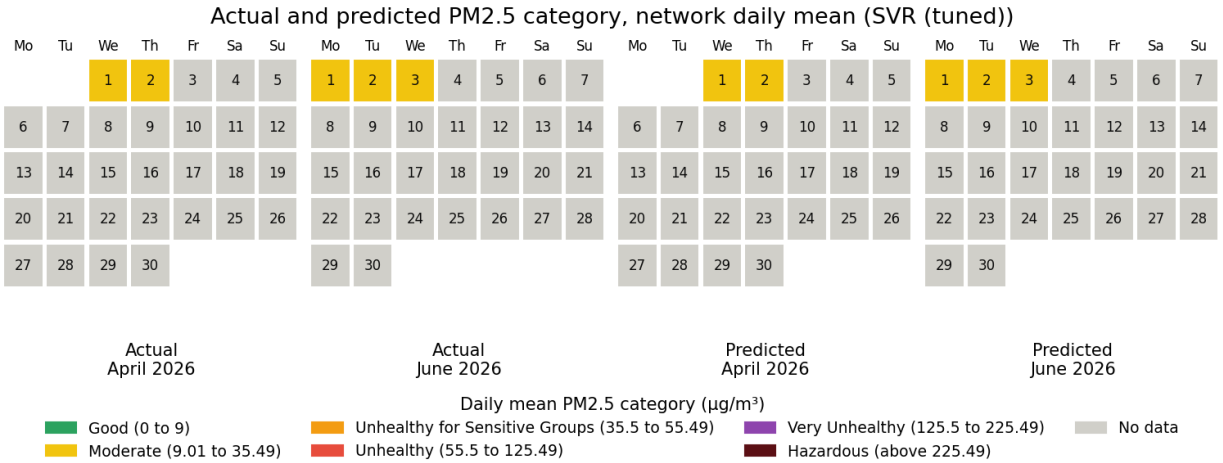


Figure 17: Actual and predicted PM2.5 category of the daily network mean (out-of-sample days only).

6.7 Data-rich against data-poor sensors

Table 11 pools every hold-out row by the sensor's data class. Every model is most accurate for data-rich sensors and least accurate for data-poor ones. The history features help data-poor sensors less, because a sensor that is often offline also often has no recent reading to use. Figure 18 shows one data-rich and three data-poor sensors.

Table 11: Hold-out MAE (μg/m^3) by sensor class.

Model	Data-rich	Intermediate	Data-poor
SVR (forecast model)	8.73	10.06	10.76
LightGBM + PM2.5 history	7.66	9.22	10.42
LSTM with PM2.5 history	10.11	11.79	12.72

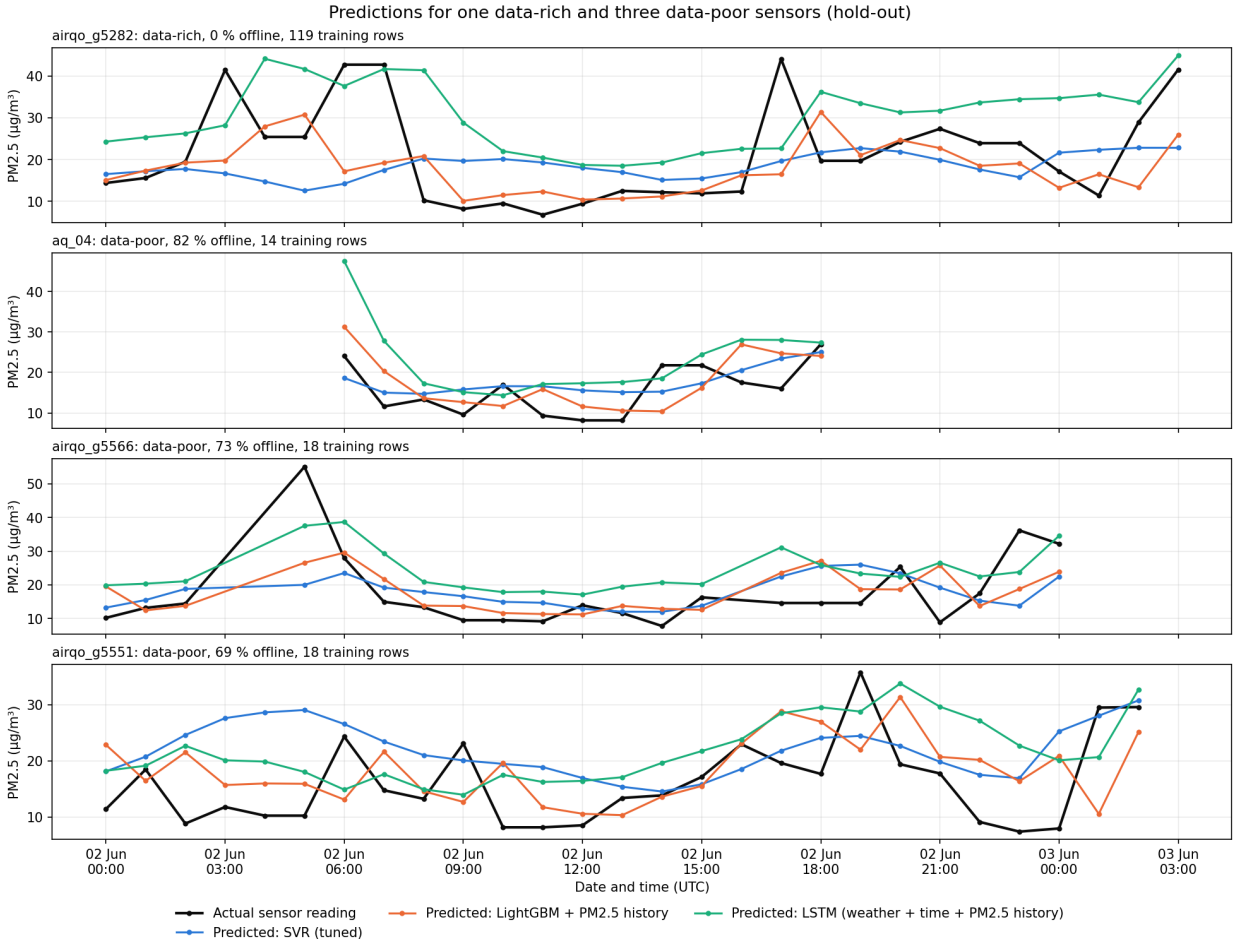


Figure 18: Predictions for one data-rich and three data-poor sensors on the hold-out.

7 Evaluation of the imputation methods

7.1 Masked-gap test

Agreement between the forward and backward estimates of the bidirectional model is a training signal, not proof of accuracy. Accuracy was measured by hiding real readings in the hold-out period (one gap per sensor at a random position, all three variables hidden, three repeats) and imputing them with each method on the same masked data (Table 12, Figure 19). A 72-hour gap cannot be tested on the main data, whose longest run of consecutive hours is 52; it is tested on the second dataset.

Table 12: Imputation MAE on hidden values by gap length. (a) is the pipeline's windowed interpolation, which does not fill gaps longer than three hours.

Variable	Method	1 h	3 h	6 h	12 h	24 h
Temperature (°C)	(a) Windowed interpolation	0.49	0.77	–	–	–
	Linear, no gap limit	0.51	0.84	1.20	1.84	2.65
	(b) Time-of-day	1.61	1.71	1.87	2.06	2.13
	(c) Bidirectional LSTM	0.76	1.13	1.36	1.74	1.97
Humidity (%)	(a) Windowed interpolation	1.80	2.66	–	–	–
	Linear, no gap limit	1.88	2.92	4.47	5.85	7.97
	(b) Time-of-day	5.91	5.94	6.76	7.40	7.99
	(c) Bidirectional LSTM	2.90	4.24	5.66	7.10	7.38
PM2.5 (μg/m^3)	(a) Windowed interpolation	7.35	9.29	–	–	–
	Linear, no gap limit	7.35	9.44	10.31	11.46	10.64
	(b) Time-of-day	9.10	10.80	10.42	10.39	12.13
	(c) Bidirectional LSTM	6.85	9.52	9.80	9.31	10.26

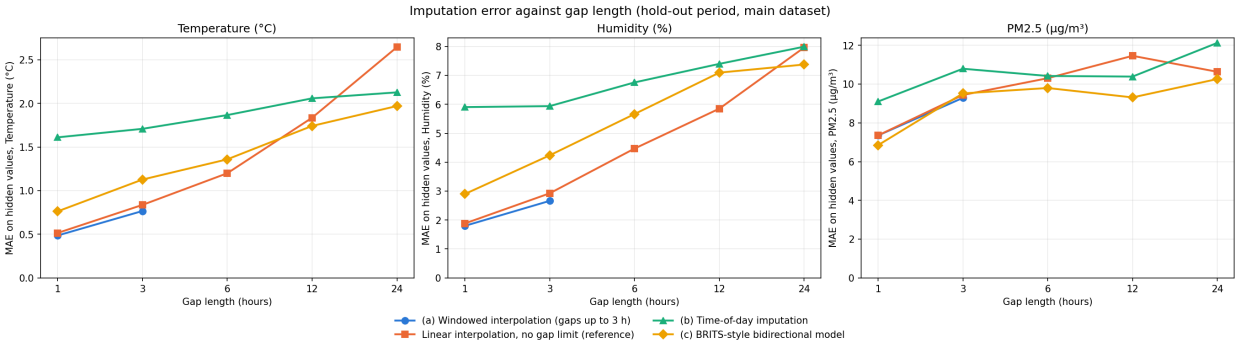


Figure 19: Imputation error against gap length for the three variables.

For gaps of one to three hours, interpolation is the most accurate method for weather. The bidirectional model is the best or joint best for PM2.5 at almost every length, and it is the best method that fills every value from six hours up. At 24 hours it is close to the time-of-day method: with no reading near the middle of a long gap its state decays and the estimate reverts to the typical daily pattern (Figure 20). PM2.5 stays hard to impute (R^2 never above 0.60, and below 0.25 from six hours on).

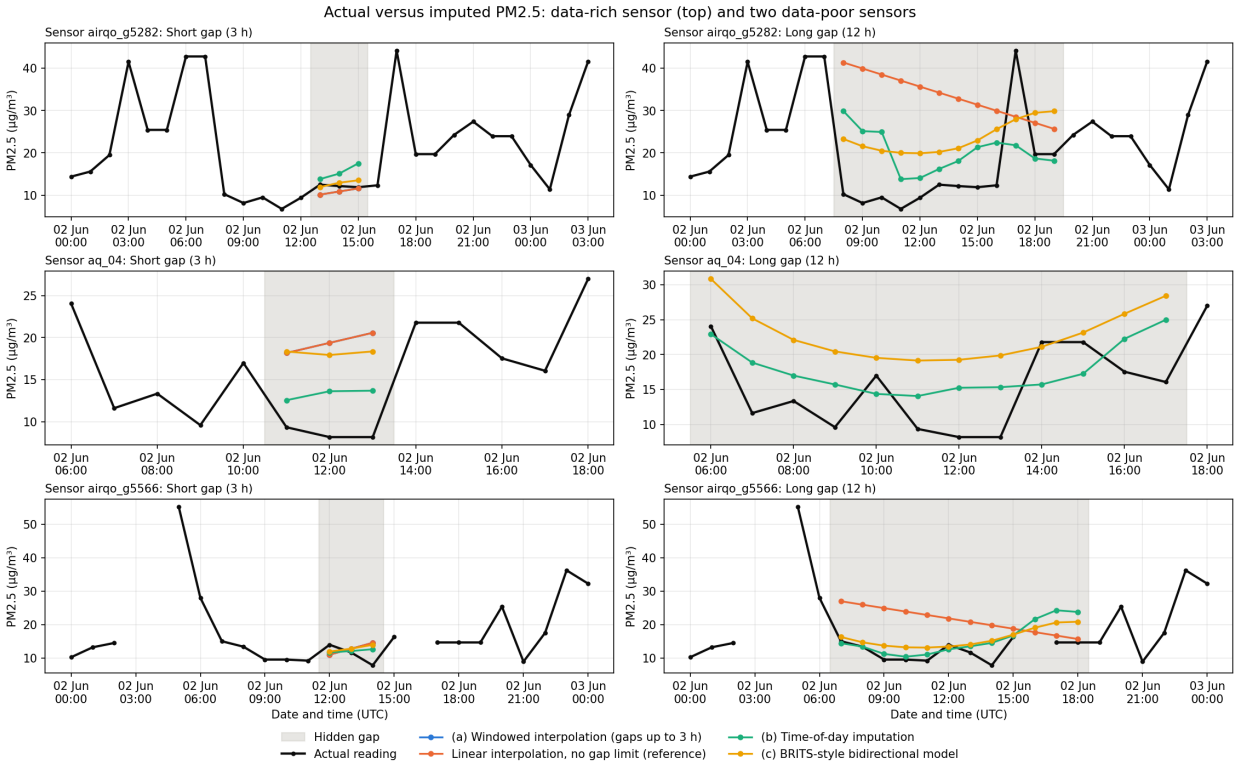


Figure 20: Actual and imputed PM2.5 for a data-rich sensor (top) and two data-poor sensors, for a short and a long gap.

7.2 Effect on PM2.5 prediction

Five models were retrained with the weather features cleaned by each method, under all three validation setups. In each fold the imputer is fitted on the training cells only, and PM2.5 is withheld from the bidirectional imputer when it fills weather so that the value being predicted cannot leak into its inputs. Table 13 gives the hold-out results; the full table for the three setups is in the notebook.

Table 13: Hold-out MAE (μg/m^3) when the weather features are cleaned by each imputation method.

Model	(a) Interpolation	(b) Time-of-day	(c) BiLSTM
SVR	9.34	9.34	9.38
LightGBM	9.94	9.82	9.74
LightGBM + PM2.5 history	8.34	8.43	8.49
LSTM (weather and time)	12.17	12.22	11.56
LSTM with PM2.5 history	10.20	10.89	9.48

The differences are small and no method is consistently best: over the fifteen model-and-setup combinations, (a) is best six times, (c) five times and (b) four times. Figure 21 shows this for all three setups: within each model the three bars are almost level, and the differences between models and between setups are far larger than the differences between imputation methods.

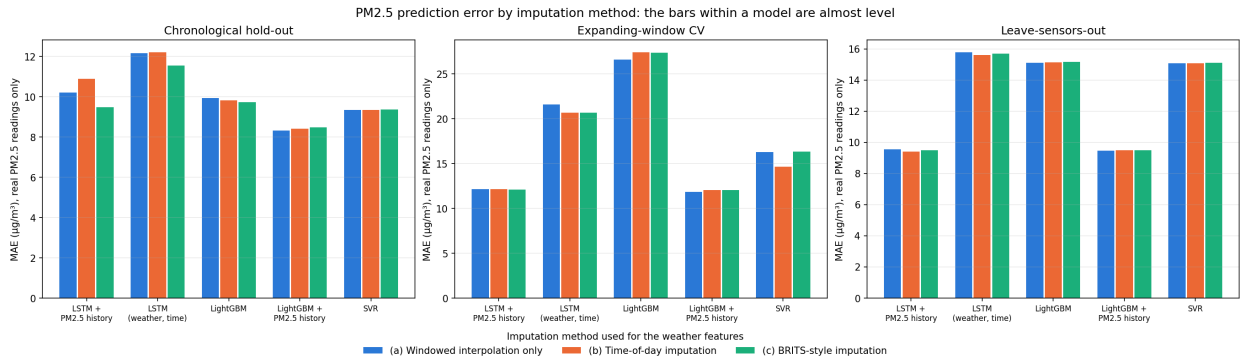


Figure 21: Prediction MAE of the five retrained models under the three validation setups, by the imputation method used for the weather features. Scores use real PM2.5 readings only.

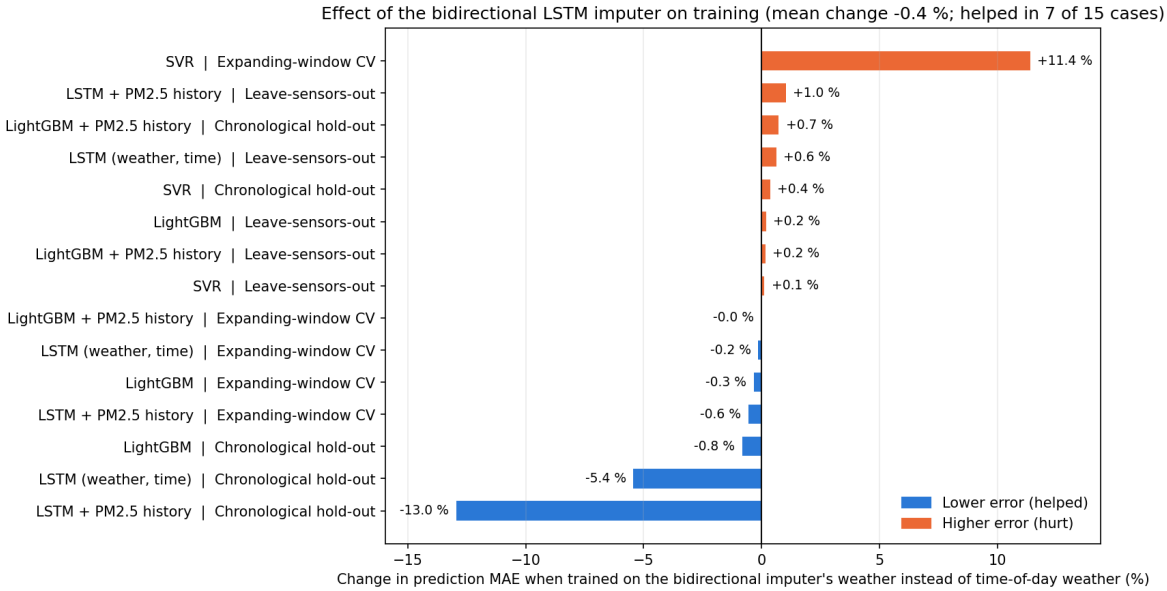


Figure 22: Change in prediction MAE when a model is trained on the bidirectional LSTM imputer's weather instead of the time-of-day weather. Negative values mean the bidirectional imputer helped.

7.3 Does the bidirectional LSTM imputation add value?

1. **As an imputer, yes.** Against the time-of-day method its error is lower in every one of the fifteen variable-and-gap combinations, by 21% on average: about half the error for one-hour weather gaps, 15 to 27% lower for temperature at 6 to 12 hours, and 6 to 25% lower for PM2.5.
2. **What it adds to the real gaps: full coverage, but smoother values.** Interpolation can fill only 5 to 12% of the values that are really missing; the time-of-day method and the bidirectional model fill all of them. The bidirectional model's fills are less varied than real readings: their standard deviation is 2.7 °C against 4.1 °C observed for temperature, and 11.9 against 26.8 μg/m^3 for PM2.5 (Figure 23). It is more accurate on average because it predicts cautiously; it does not reproduce peaks.
3. **For training, no reliable gain.** Training on its weather instead of the time-of-day weather changed the prediction MAE by −0.4% on average and helped in 7 of 15 cases. It helped the LSTMs on the hold-out (−13% and −5%), hurt the SVR in the expanding-window CV

(+11%), and changed everything else by about 1% or less (Figure 22).

The bidirectional imputer is the right tool when the imputed values themselves are the product, especially for gaps of 6 to 12 hours. For training the PM2.5 models here it brings no reliable gain over the simpler time-of-day method, which the pipeline keeps, because weather is a minor input for these models (Section 8).

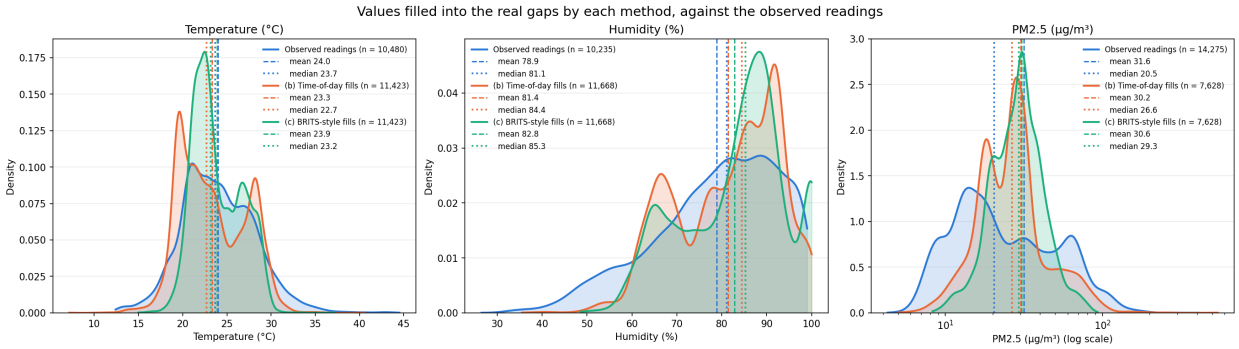


Figure 23: Values filled into the real gaps by the time-of-day method and by the bidirectional model, against the observed readings.

7.4 Decision: which imputed data the final models are trained on

The final models, including the deployed forecast model and the models with PM2.5 history, are trained on weather filled by windowed interpolation followed by time-of-day imputation. The bidirectional LSTM imputer is kept in the notebook as a tested method, but its values are not used to train the final models, and the remaining models were not retrained on it. The reasons are:

- **The test showed no reliable benefit.** Across five models and three validation setups the prediction MAE changed by -0.4% on average and improved in 7 of 15 cases (Figures 21 and 22).
- **There is little for it to improve.** The imputer fills only temperature and humidity, which carry a small share of what the models rely on (Section 8).
- **The one clear gain does not change the outcome.** It lowered the hold-out MAE of the LSTMs (10.89 to 9.48 $\mu g/m^3$ with PM2.5 history, 12.22 to 11.56 $\mu g/m^3$ without), but they remain behind the tree models with PM2.5 history (8.43 $\mu g/m^3$), so the best model and the deployed model would be the same.
- **Cost and simplicity.** The bidirectional imputer must be retrained for every data split and would add a neural network to the serving path. The time-of-day method needs no training and is easy to explain.
- **The untested models use the same inputs.** Ridge, Linear, Polynomial, KNN and the Random Forests use the same two weather features, so a different result from retraining them is not expected. This was not tested.

Independently of this decision, PM2.5 is never imputed for training or scoring. The decision should be revisited if weather becomes a stronger predictor (for example with forecast weather for multi-day forecasting), if gaps of 6 to 12 hours become common in the data being served, or if the

imputed series itself is the deliverable; in those cases the masked-gap test shows the bidirectional imputer is the better choice.

8 Model explainability with SHAP

SHAP [10] splits each prediction into a contribution from every input feature. TreeExplainer [11] was used for the tree models and GradientExplainer for the LSTM. Values were computed on 400 rows of the test split only, with 100 training rows as background, in $\mu g/m^3$, with the 149 sensor columns grouped into one feature. SHAP shows what a model relies on; it does not show what causes PM2.5.

Table 14: Share of mean absolute SHAP value by feature group (test split).

Feature group	Random Forest	LightGBM	LightGBM + PM2.5 history
Recent PM2.5 (history)	–	–	50%
Month	46%	32%	11%
Hour of day	18%	19%	12%
Sensor identity	18%	27%	11%
Day of week	5%	10%	7%
Temperature	8%	7%	7%
Humidity	5%	6%	3%

Without PM2.5 history, the tree models rely on when and where a reading was taken, not on the weather (Table 14, Figure 24). The weight on month is a warning: the training data hold six observation days in three months, so the month mostly tells the model which burst a row came from, which does not transfer to other dates. With PM2.5 history, half of the attribution moves to the recent readings and the model stops leaning on the calendar (Figure 25); this is the mechanism behind its better and more stable scores. The LSTM agrees: the previous hour's PM2.5 is its most important input, and most of the attribution sits in the last step of its window (Figure 26).

Weather carries about the same share of attribution whether it was measured or imputed, so imputed values are not driving the predictions. The local explanations show the cost of having no signal for episodes: the highest reading in the test split ($172\,\mu g/m^3$) is predicted as $33\,\mu g/m^3$.

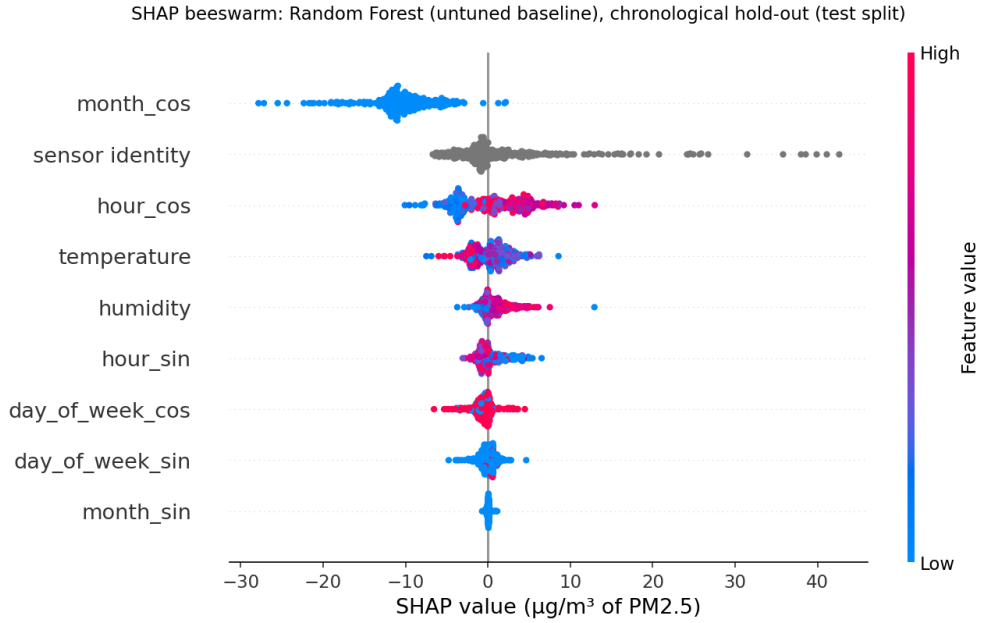


Figure 24: SHAP beeswarm for the Random Forest without PM2.5 history (test split).

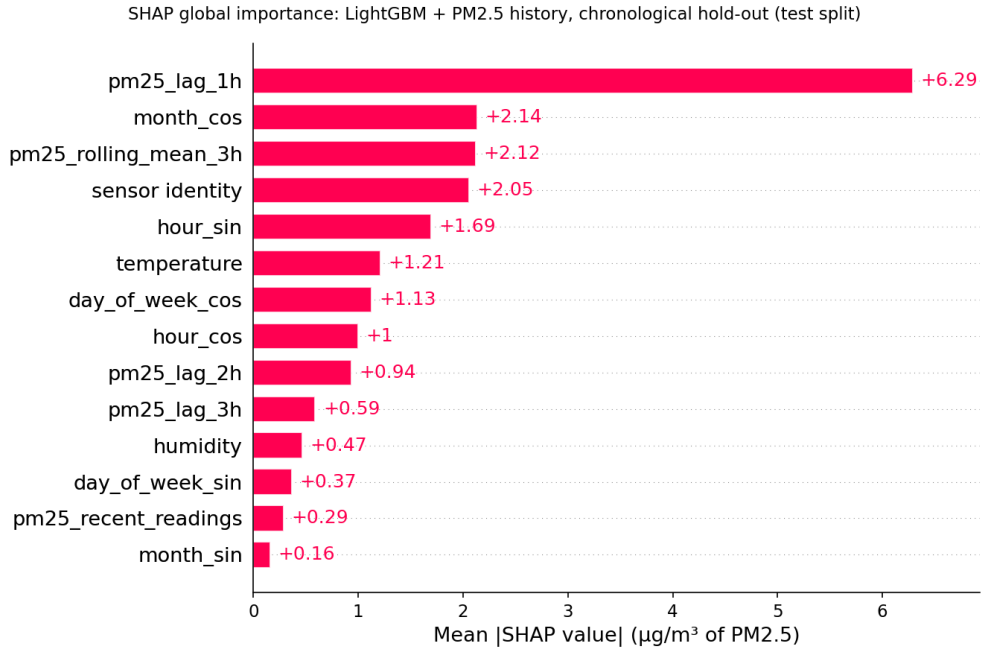


Figure 25: SHAP global importance for LightGBM with PM2.5 history (test split).

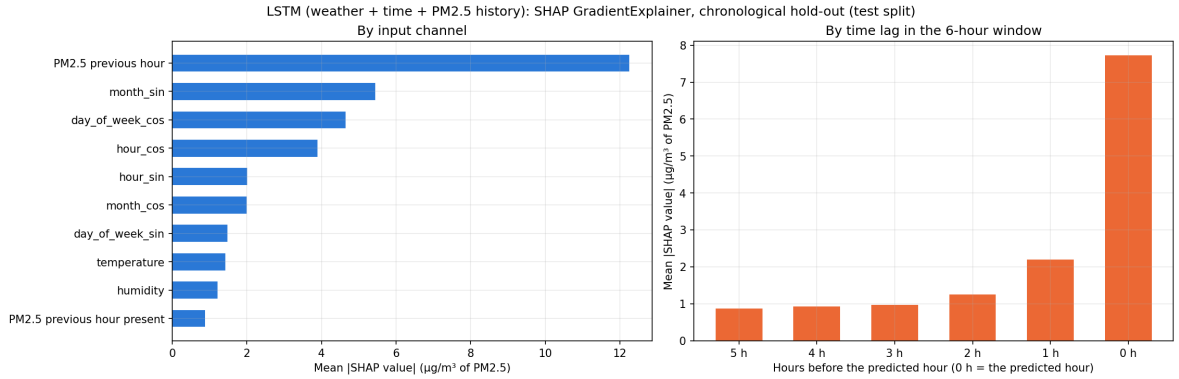


Figure 26: LSTM with PM2.5 history: attribution by input channel and by time lag.

9 Sensor maps

The data contain latitude and longitude. The median coordinate per sensor was used; 143 of the 149 sensors have coordinates and the other six are left off the maps. Points were built with `shapely` and held in a `geopandas` GeoDataFrame in WGS84, reprojected to UTM zone 36N for distances. Regional boundaries come from geoBoundaries (OpenStreetMap, Open Database License); no web tile service is used.

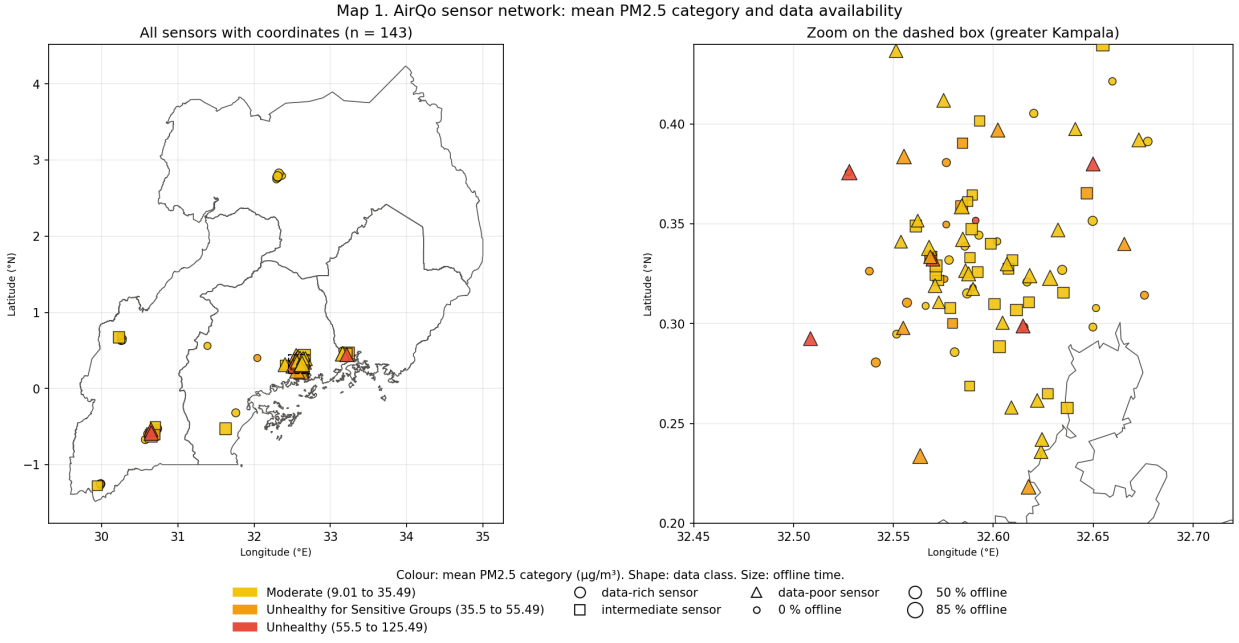


Figure 27: Sensor network: colour is the PM2.5 category of the mean reading, shape the data class, and size the offline time.

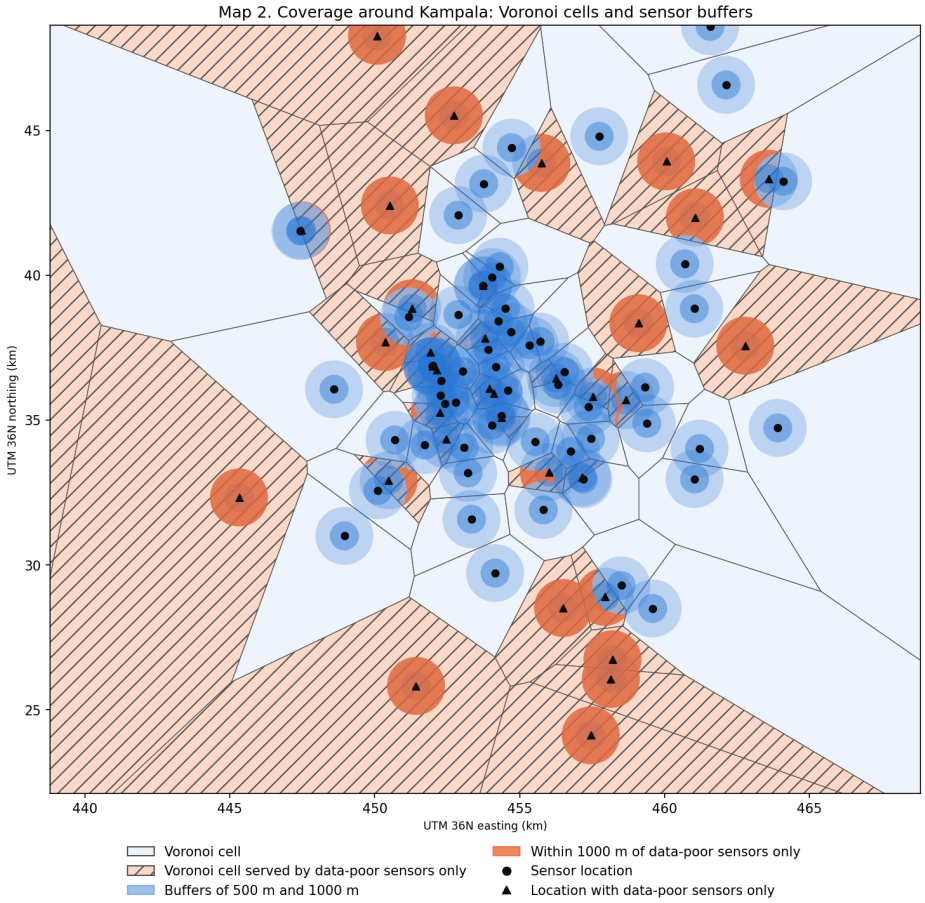


Figure 28: Coverage around Kampala: Voronoi cells clipped to the network's convex hull, and buffers of 500 m and 1 km. Hatched cells are served by data-poor sensors only.

Of 140 distinct sensor locations, 41 have data-poor sensors only. The area within 1 km of any sensor is 297 km^2, of which 60 km^2 is within 1 km of data-poor sensors only (Figure 28).

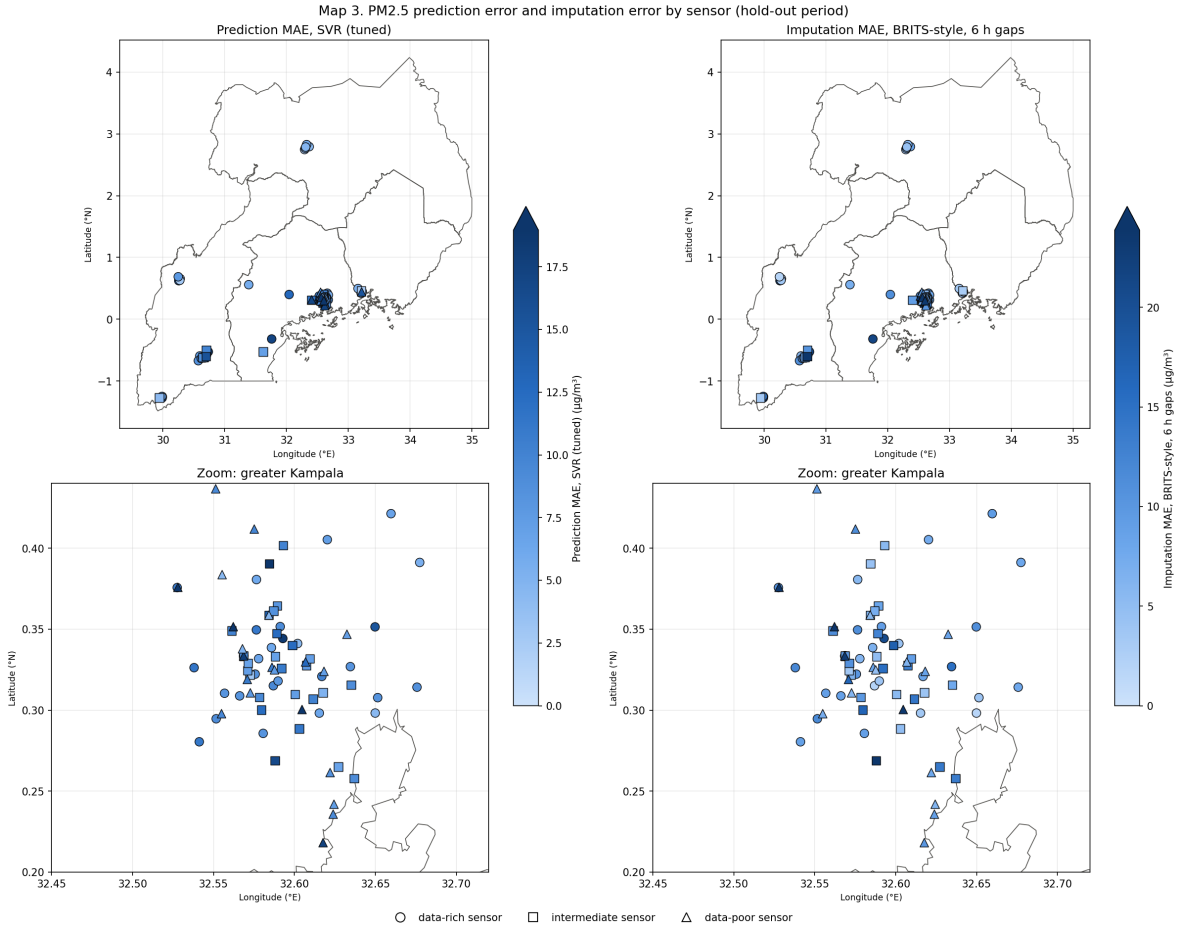


Figure 29: Prediction error of the selected model and imputation error of the bidirectional model, by sensor.

Error does not follow sensor density: its rank correlation with the distance to the nearest neighbour is −0.08 (prediction) and −0.07 (imputation). The clearest link is with how polluted the site is (0.43 and 0.31); offline time has a weaker link (0.22 and 0.19).

10 Second dataset test

The file `AirQo_August_2026_hourly_matched.csv` was used as an external test set with no retraining. It has 73,274 hourly rows from 125 sensors for 1–31 August 2026, with the same columns and units as the main file. All 125 sensors are also in the main dataset and all are in Uganda. Its PM10 column was dropped and its zero temperature and humidity readings (about 14%) were treated as missing, as in the main data. August was more polluted than the hold-out days (mean PM2.5 35.5 against 23 μg/m^3). The extreme-value replacement was not applied to this file, so the models are scored against the original August readings.

Table 15: PM2.5 prediction on the second dataset against the main hold-out (μg/m^3).

	August 2026			Hold-out
Model	MAE	RMSE	R^2	MAE
Random Forest + PM2.5 history	13.70	21.71	0.30	8.50
LightGBM + PM2.5 history	14.02	22.14	0.27	8.43
SVR (selected forecast model)	18.32	27.95	−0.16	9.34
KNN	18.65	27.58	−0.13	10.01
LightGBM	18.79	27.94	−0.16	9.82
Random Forest (untuned)	19.05	28.67	−0.22	9.61
Baseline (training mean)	19.21	26.04	−0.01	17.60
LSTM with PM2.5 history	19.31	25.59	0.03	10.89
LSTM (weather and time)	19.47	26.61	−0.05	12.22

Generalisation. The forecast models do not generalise to a new month: they are barely better than always predicting the training mean (Table 15), and the forecast model predicts Moderate on all 31 days while the actual daily mean is Unhealthy for Sensitive Groups on 16 of them (Figure 30). The tree models with PM2.5 history keep part of their skill (MAE 13.7 and 14.0 μg/m^3, R^2 about 0.3) and match the category on 17 of 31 days. Every sensor in this file was seen in training, so unfamiliar sensors are not the explanation: the models know these sensors, but only at their January, April and June levels.

The likely reasons for the drop are an unseen month (month is the largest SHAP driver of the forecast trees), a more polluted period, and too little training data in time: six training days in three bursts cannot teach a model what a full month looks like. Evidently flags 6 of 9 input features as drifted between the training data and August (Figure 35). Missingness is not a cause: 76 of the 125 sensors are data-rich in August. Table 16 shows that the data-poor sensors have the lowest absolute error, because their sites were cleaner in August; relative to the mean level the three classes are alike.

Table 16: Second dataset: MAE (μg/m^3) by sensor class, and the mean actual PM2.5 of each class.

	Data-rich	Intermediate	Data-poor
Sensors	76	31	18
Mean actual PM2.5	36.0	35.3	29.8
SVR (forecast model)	18.72	18.08	14.32
LightGBM + PM2.5 history	14.04	14.36	12.51
LSTM with PM2.5 history	19.04	19.93	20.24

Imputation generalises well. With the imputers fitted on the main training period and applied unchanged, the bidirectional model is as accurate on August as on the main hold-out for temperature (1.3 °C at six hours and 1.8 °C at 24 hours) and holds at 72 hours (2.0 °C, the same as the time-of-day method). For PM2.5 it is the best method at every gap length from six hours up (13.9 to 16.7 μg/m^3 against 15.7 to 18.4 μg/m^3 for the time-of-day method). The daily cycle of weather transfers to a new month; PM2.5 levels do not, unless the model is given recent readings.

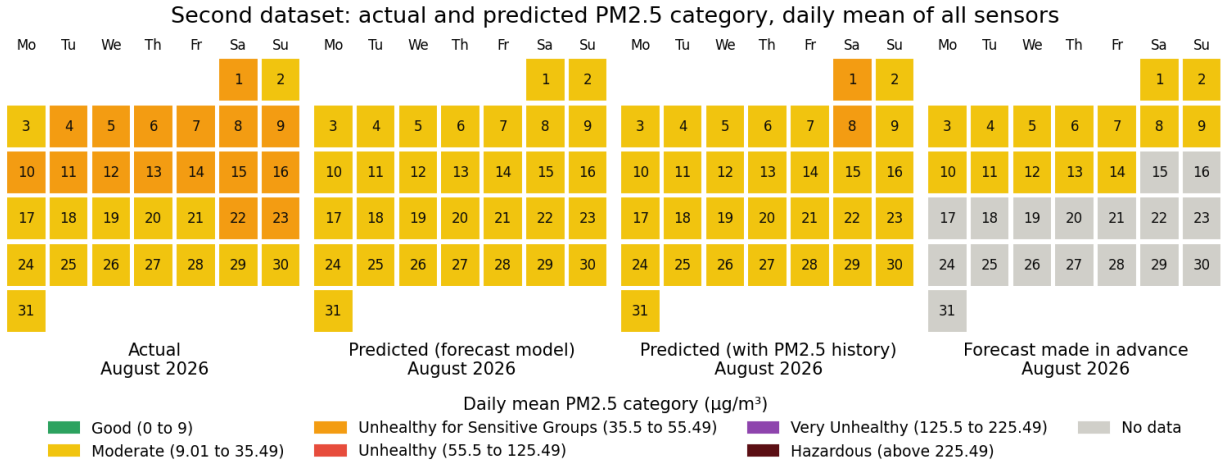


Figure 30: Second dataset: actual and predicted PM2.5 category of the daily mean, and the forecast made in advance for 1–14 August.

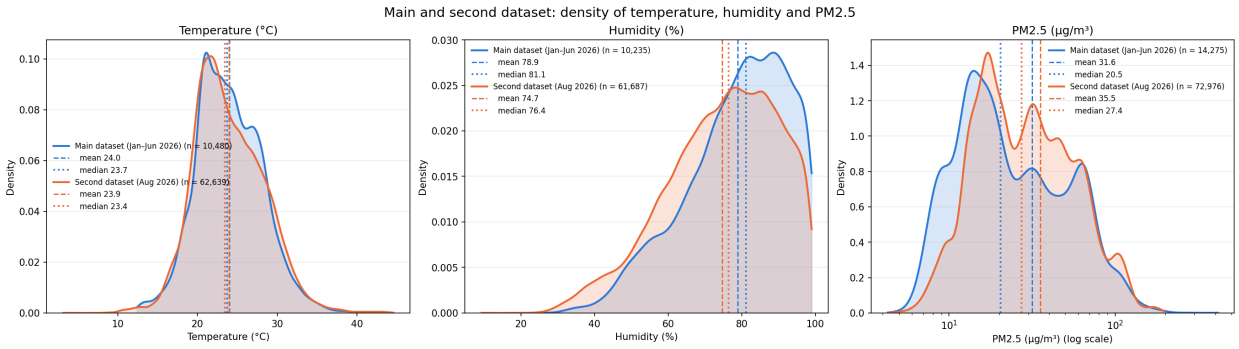


Figure 31: Main and second dataset: density of temperature, humidity and PM2.5.

11 Forecasting and visualisation

The saved forecast model (SVR) produced daily forecasts for all 149 sensors for 1–14 August 2026 (2,086 sensor-days) and hourly forecasts for 1 August 2026 (3,576 sensor-hours). August lies outside the main data, so the time features are computed from the forecast timestamps, while temperature and humidity are each sensor's typical value for that hour of day. All forecast checks passed: correct row counts, all sensors and days, and no missing or negative predictions.

Of the 2,086 sensor-days, 2,082 are forecast as Moderate and four as Unhealthy for Sensitive Groups (Figure 32). The forecasts repeat each sensor's typical pattern; they do not react to actual conditions. The second dataset shows how far that is from reality: the actual August daily mean was above Moderate on 16 of 31 days. Reliable forecasts beyond a few hours would need recent readings or forecast weather as inputs.

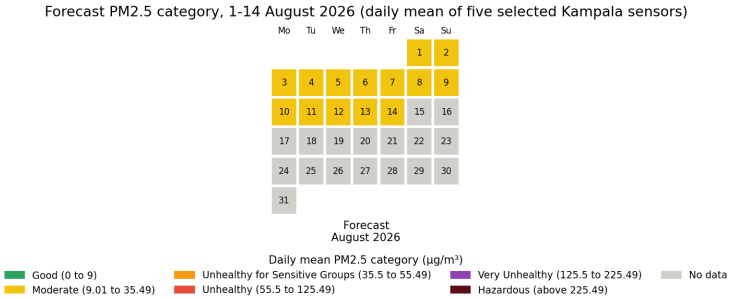


Figure 32: Forecast PM2.5 category for 1–14 August 2026 (daily mean of the five Kampala sensors).

12 Model monitoring with Evidently AI

Evidently 0.7.23 [17] compares a *reference* dataset, the training period (11,633 rows), with a *current* dataset, the test period (2,642 rows). The Evidently cells are wrapped so that an Evidently error cannot crash the pipeline, as the brief requires.

Phase 1 monitoring. In the first pipeline Evidently flagged 6 of 7 scored columns as drifted. Those seven were the time features and the sensor identity; temperature, humidity and PM10 were not scored, most likely because they were held in the nullable `Float64` type, which that report did not pick up. The drift came from the make-up of the two windows (different months, weekdays and active sensors), not from a change in the air. In the revised pipeline temperature and humidity are among the nine scored features below.

Data drift. For each feature Evidently reports a drift score: the normed Wasserstein distance for numeric features and the Jensen–Shannon distance for categorical or low-cardinality features, with a threshold of 0.1. Eight of nine features drifted (share 0.89). The largest scores are day of week (1.43 and 1.19) and month (0.65), because the test window holds only two weekdays of one month; temperature and humidity drifted moderately (0.36 and 0.37) and hour of day barely (0.08 and 0.15). The alert means “the current data is not representative of what the model was trained on” rather than “pollution behaviour has changed” (Figure 33).

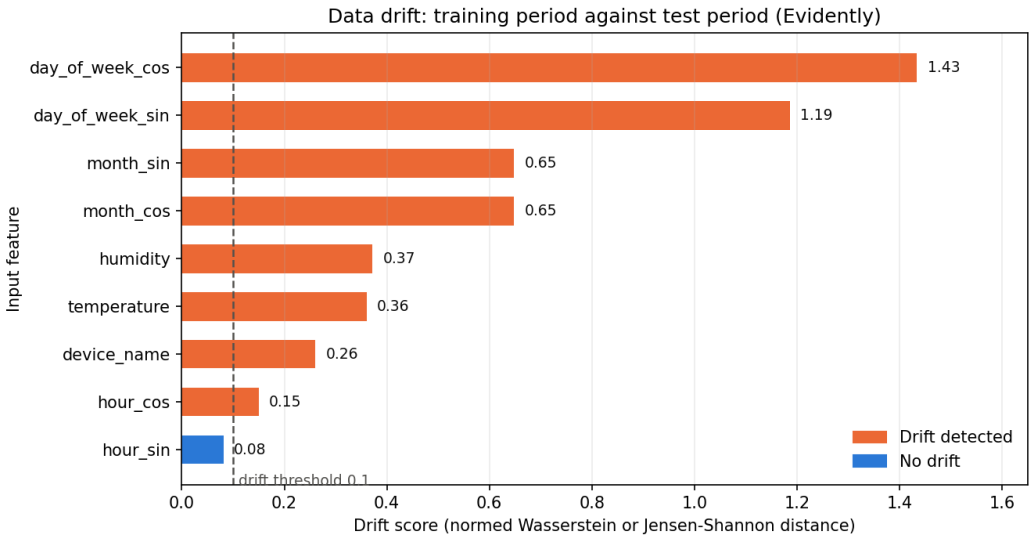


Figure 33: Evidently data-drift scores, training period against test period. Orange bars exceed the 0.1 threshold.

Model drift. Table 17 and Figure 34 compare the selected forecast model on the two periods. Error rises from reference to current; part of that is expected because reference predictions are in-sample. The mean prediction ($22.4\,\mu g/m^3$) stays close to the mean actual value ($23.4\,\mu g/m^3$), so the model is not systematically biased on the test period.

Table 17: Performance of the selected forecast model, reference (training) period against current (test) period.

Period	MAE ($\mu g/m^3$)	RMSE ($\mu g/m^3$)	R^2
Reference (training, in-sample)	5.90	12.10	0.81
Current (test)	9.34	15.14	0.34

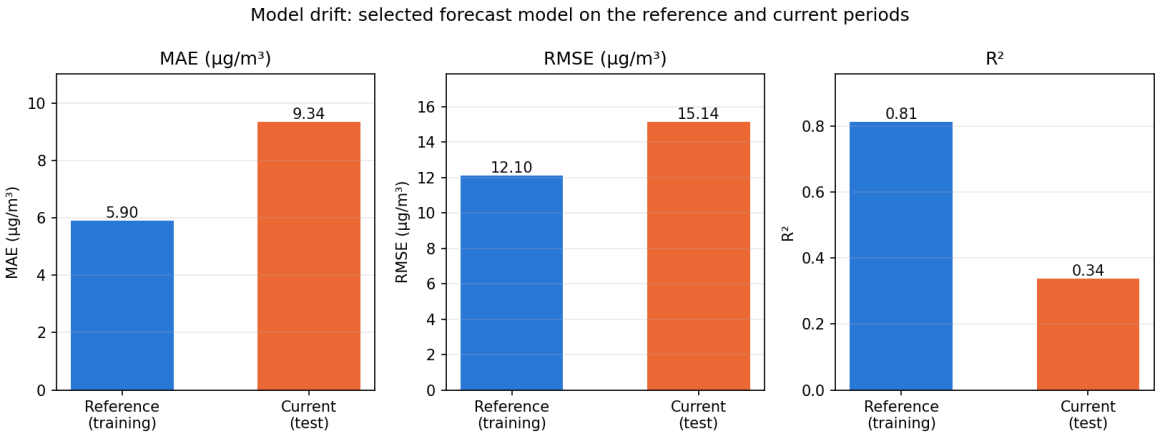


Figure 34: Model drift: MAE, RMSE and R^2 of the selected forecast model on the reference (training) and current (test) periods.

The mean error on the test period is $-0.97\,\mu g/m^3$ with a standard deviation of $15.1\,\mu g/m^3$, the mean absolute percentage error is 44.9%, and the largest single error is $134\,\mu g/m^3$. The model

is therefore close to unbiased on average but can miss individual hours by a wide margin, which matches the residual plots in Section 6.5.

Drift against the August dataset. A second drift report compares the training data with the August dataset (Figure 35). Six of nine features drifted. Month drifts most (0.83), because August was never seen in training, followed by day of week (0.75 and 0.61), humidity (0.26) and the mix of sensors (0.25); temperature and hour of day did not drift. This is the input-side explanation of the weak August results in Section 10: the model is being asked about a month outside its training data. In production, a drift share this high together with a rising error on newly labelled readings would trigger retraining.

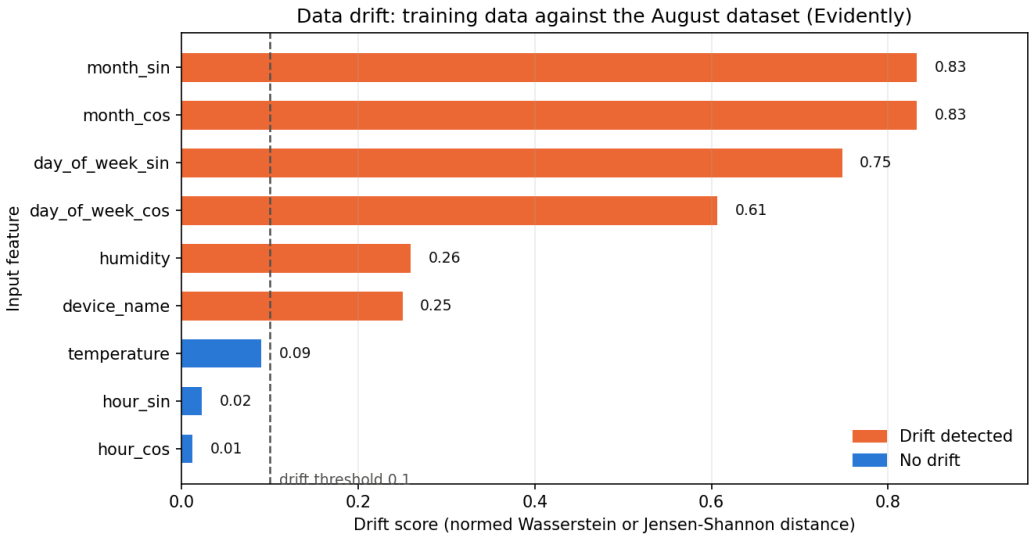


Figure 35: Evidently data-drift scores, training data against the August dataset.

13 Deployment

The pipeline is deployed in three parts: the training pipeline is the notebook, verified end to end (Section 13.2); the trained model is served as a web service on **Render**; and a dashboard that uses that service is hosted on **Vercel**.

13.1 Deployment history and choice of platforms

The deployment changed several times. In order:

1. **RunPod (planned).** The brief proposed RunPod, a pay-as-you-go GPU cloud. We prepared to run the pipeline on an RTX 2000 Ada Pod with a run script and a prediction endpoint. RunPod requires credit to be bought in advance, which became financially straining for the two of us, so this plan was dropped.
2. **Kaggle (cloud execution).** The first, PM10-based version of the notebook was run end to end on Kaggle's cloud notebook service as the evidence of cloud execution, with a run-log section that records the runtime, hardware, row counts, metrics and output files.
3. **Hugging Face (model and static demo).** The first model was published as the model

repository `Kyalis/airqo-pm25-random-forest`, and its August forecasts as a static demo Space, `Kyalis/airqo-pm25-forecasts`. Server-side Spaces (Gradio, Docker or Streamlit) were refused with HTTP 402 because they need a paid PRO plan, so the demo only displays stored forecasts and does not run the model. Running the model in the browser with ONNX would avoid this and is left as future work.

4. **Render and Vercel (final).** For the revised pipeline we wanted the model itself to answer requests. The model runs as a web service on Render and the dashboard is hosted on Vercel (Sections 13.3 and 13.4).

Render and Vercel both have free tiers and fit the two halves of the problem:

- **Render** runs a long-lived Python web service, which is what loading a scikit-learn model and predicting on request needs.
- **Vercel** hosts the front end, a web dashboard that calls the Render service from the visitor's browser.

13.2 Local verification

The revised pipeline was first run cleanly end to end locally with the pinned package versions in `requirements.txt` (Table 18).

Table 18: Local verification of the revised pipeline (Windows 11, Python 3.13, 16 logical CPUs).

Check	Result
Full notebook run	346 code cells executed, 0 errors, 70 figures
Pipeline runtime	3,357 s (about 56 minutes), CPU only
Peak RAM	1,230 MB
Forecast model artifact	1.2 MB (SVR pipeline, target scaler, feature lists)
Nowcast model artifact	0.6 MB (LightGBM with PM2.5 history)
Prediction latency (forecast model)	1 row: 14 ms; 3,576 rows: 1.4 s; 50,064 rows: 20.7 s
Standalone inference script	Identical predictions to the notebook on all 2,642 test rows

The runtime is much longer than in the earlier version (11 minutes) because every model is refitted for seven validation folds, the bidirectional imputer is retrained per fold, and the second-dataset test, SHAP and maps were added. Training needs `lightgbm`, `imbalanced-learn`, `tensorflow`, `shap`, `geopandas` and `shapely` in addition to the original packages.

13.3 Model service on Render

The saved forecast model (`best_pm25_model.joblib`: the SVR pipeline with its preprocessing, the target scaler and the feature lists) is served by a small Python web service on Render at **[ADD LINK: Render service URL]**.

- **Small footprint.** The service only needs the packages required to load the model and predict. They are pinned in `requirements-render.txt`: `numpy`, `pandas`, `scikit-learn`, `scipy` and `joblib`. TensorFlow, LightGBM, SHAP and the mapping libraries are needed for

training and analysis only, so they are not installed on the server. The model file is 1.2 MB and a single prediction takes about 10 ms, which suits a free instance.

- **Interface.** The service accepts a JSON `POST` request to `/predict` with the place (sensor), the date and time in UTC, the temperature and the humidity, and returns the predicted PM2.5 in μg/m^3 together with its AQI category. The time features (hour, day of week and month as sine and cosine pairs) are derived from the timestamp, exactly as in training, and the prediction is transformed back from the Z-score scale with the saved target scaler.
- **Same code path as the notebook.** The service uses the serialised pipeline unchanged, so preprocessing is identical to training. The standalone inference script reproduced the notebook's predictions exactly on all 2,642 test rows.

13.4 Dashboard on Vercel

The dashboard is hosted on Vercel at **[ADD LINK: Vercel dashboard URL]**. It shows model estimates of hourly PM2.5 for the AirQo sensors for August 2026 and reports whether the model service is connected.

- **Filters and summary** (Figure 36). The user chooses a place, a date range and an hourly or daily resolution. The page shows the mean, maximum and minimum predicted PM2.5 with their AQI categories, and the share of time in each category. Every panel states what the inputs are: for the place shown, 97% of hours use observed August temperature and humidity and 3% use the place's typical hourly profile, which is labelled as assumed.
- **Predicted against actual** (Figure 37). The predicted series is drawn over the AQI category bands, with the actual sensor readings where they exist and the MAE and RMSE against them for the selected range.
- **Calendar and map** (Figure 38). A daily calendar of the mean predicted PM2.5 in the category colours, and a map of the places coloured by their mean predicted category. The 143 of 149 places that have coordinates in the source data are shown.
- **Try the model.** A form sends a place, date and time, temperature and humidity to the service's `/predict` endpoint and shows the prediction, its category and the exact request that was sent.

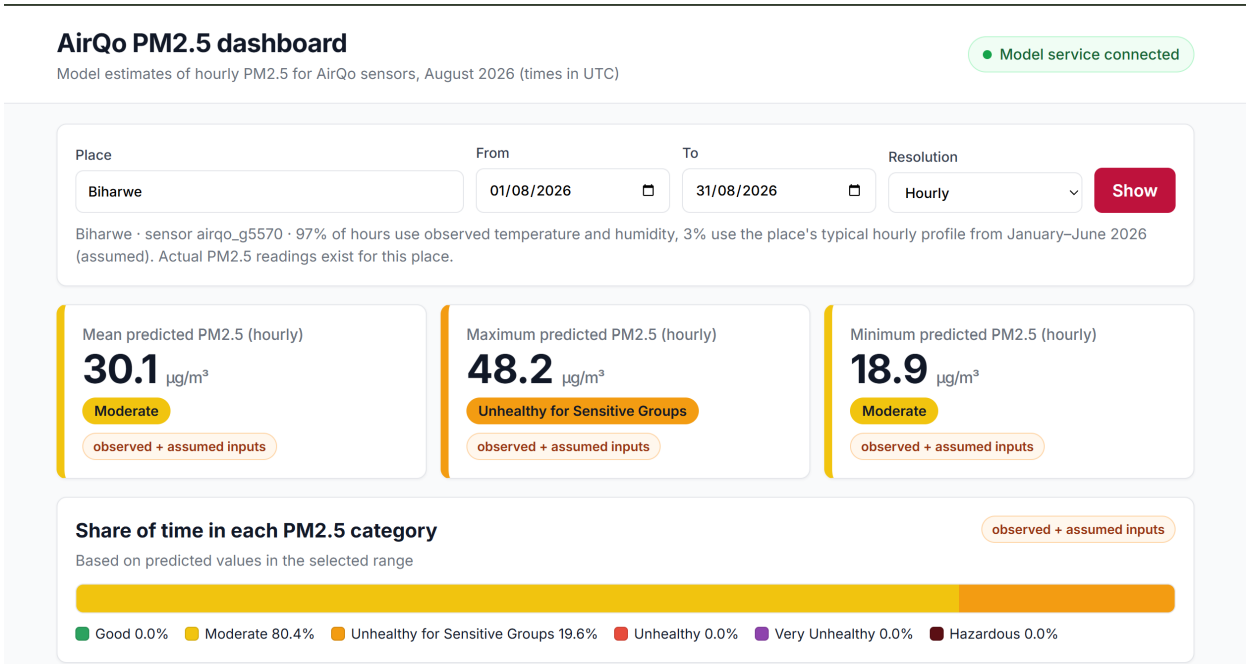


Figure 36: Dashboard: filters, summary of predicted PM2.5 and share of time in each category.

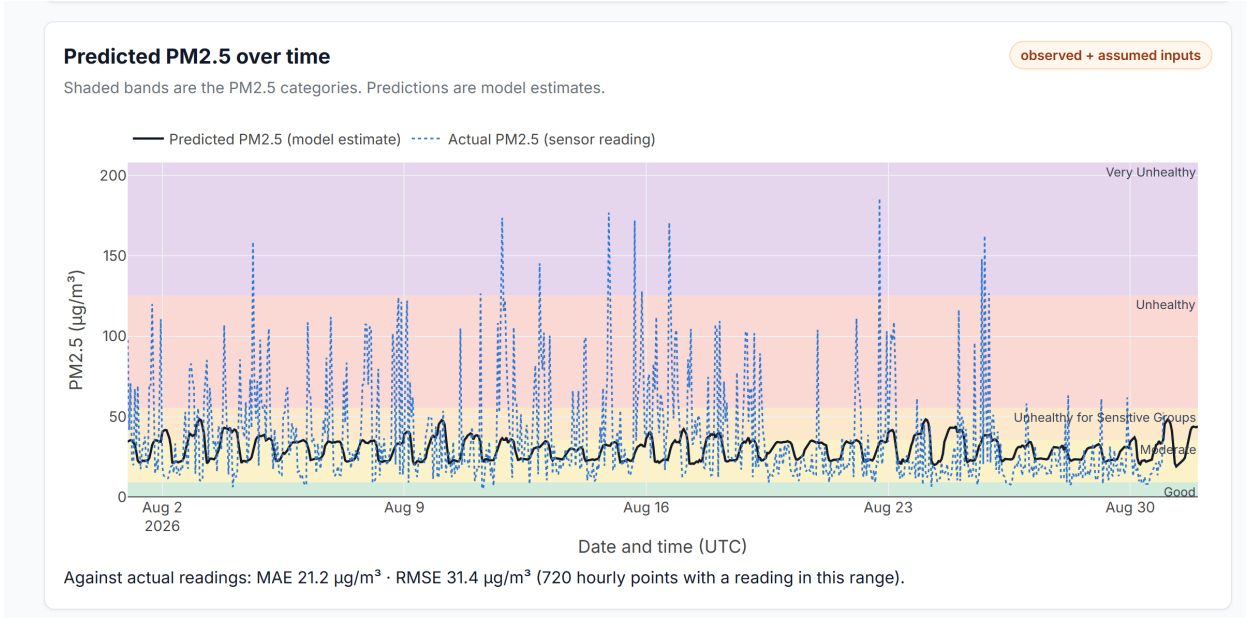


Figure 37: Dashboard: predicted PM2.5 over the AQI category bands, with the actual sensor readings and the error against them.

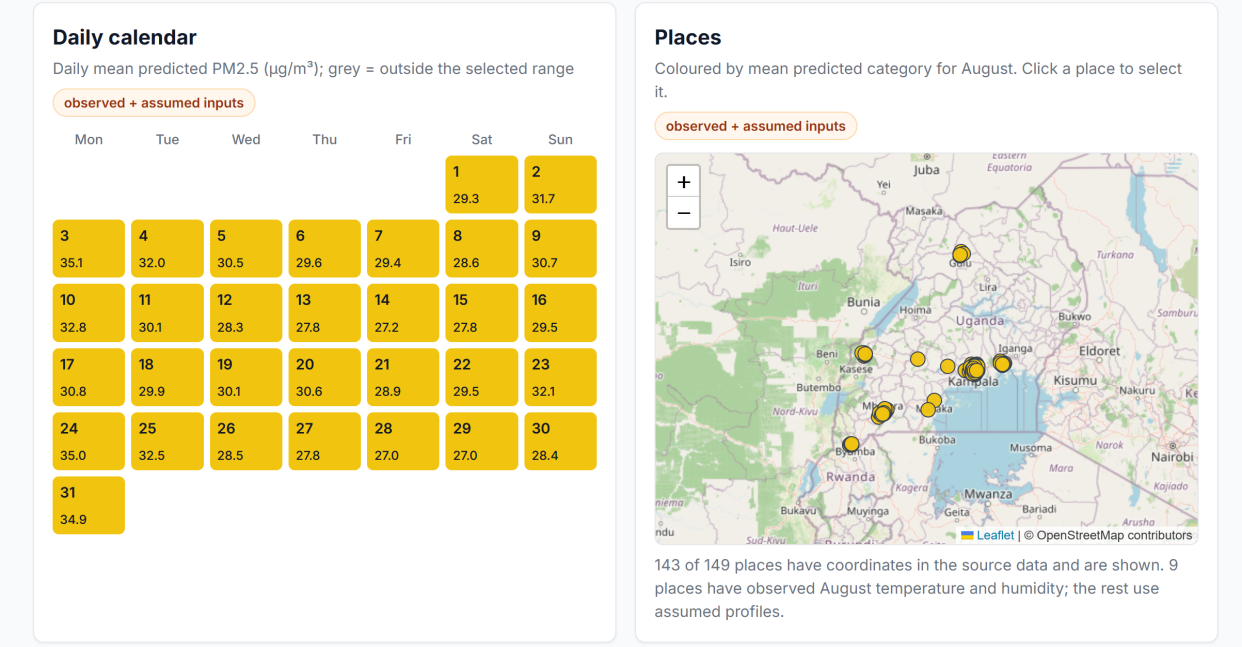


Figure 38: Dashboard: daily calendar of predicted PM2.5 and map of the places.

What the deployment shows about the model. The dashboard is honest about the model's limits. For the place in Figure 37 the predicted line stays between about 20 and 50 μg/m^3 while the actual readings spike above 150 μg/m^3, and the error against the actual August readings is MAE 21.2 μg/m^3 (RMSE 31.4 μg/m^3). That agrees with the external test in Section 10: the forecast model predicts a typical daily pattern and does not anticipate episodes.

13.5 Limitations of the deployment

- **Forecast model only.** The service runs the forecast model (SVR). The more accurate model with PM2.5 history is saved but not served, because it needs the sensor's readings from the previous three hours with every request.
- **Free tiers.** A free Render instance stops when idle and takes time to start again on the first request, and it has limited memory, which is why the service installs only the small set of packages above.
- **No automatic retraining.** The service loads a fixed model file. Retraining still means running the notebook and redeploying the new file.
- **Earlier deployments.** The Kaggle run and the Hugging Face model card and static demo were made with the first, PM10-based version of the pipeline and do not reflect the revised model.

14 Discussion and limitations

- **Little data in time.** Eight days in three bursts cannot capture seasonal patterns. The three validation setups disagree on the ranking of the forecast models, and the external August test, on 125 of the same sensors, shows that the forecast models do not transfer to a new month.

- **Short test window.** The chronological hold-out is two days (2–3 June); the expanding-window and leave-sensors-out setups and the August test are needed to judge stability.
- **Weak predictors without PM10.** With PM10 the earlier Random Forest scored MAE 3.10 μg/m^3 (R^2 0.90), but that score came from a co-measurement of the same particles, read in the same hour, rather than from what drives PM2.5 (Section 1.2). PM10 is also the strongest predictor in Gürtepe et al. [4], which shows how closely the two measurements track each other; it does not make PM10 a cause of PM2.5. Removing it is the right choice for forecasting, but on our data the drivers that remain explain little: temperature and humidity carry only 3–8% of the SHAP attribution (Table 14). Eight days are too few for the model to learn the weather effects that the literature describes, and the recent PM2.5 readings are the only strong signal available.
- **Sensor calibration.** PM2.5 exceeded PM10 in 22.4% of the raw rows and some PM10 readings were below 1 μg/m^3, which point to calibration problems in the low-cost sensors. The models learn from these readings as they are, apart from the extreme-value replacement.
- **Forecast and nowcast are different tasks.** The models with PM2.5 history are the most accurate but need recent readings from the sensor, so they cannot produce the two-week forecast.
- **Extreme values.** Replacing extreme readings lowers the reported errors; scored against the untouched readings the best models' MAEs are about 1.3 μg/m^3 higher. The replacement rule uses each sensor's whole record, and it was not applied to the second dataset.
- **SMOTE** was evaluated and rejected on evidence; the cross-validation used to choose its strength was itself unstable because the folds come from different bursts.
- **Imputation.** The bidirectional imputer's PM2.5 fills are too smooth to serve as training targets, and its benefit for prediction is negligible on this data. Only five models were retrained on its values; Ridge, Linear, Polynomial, KNN and the Random Forests were not retested, so the conclusion for them is an expectation, not a measurement.
- **Deployment.** The live service runs the forecast model only, on free hosting tiers; the model with PM2.5 history is not served.

14.1 Future development

- **More days of data.** The largest improvement would come from training on continuous data, such as the August file, instead of three two-day bursts.
- **Forecasting from history.** Combine lagged pollutant features with multi-step forecasting, feeding each prediction forward, and add forecast weather.
- **Neighbouring sensors.** Use same-hour PM2.5 from nearby sensors, which the maps show are dense around Kampala.
- **Managed cloud deployment.** Move the model service to Google Cloud (Cloud Run or a Vertex AI endpoint) or Azure (Azure Machine Learning online endpoints or Container Apps), which add autoscaling and model versioning.
- **In-browser inference.** Export the model to ONNX so that a static page, such as the Hugging Face Space, can run it without a server.
- **Automated monitoring**, with scheduled Evidently reports, retraining when error grows,

and rollback to the previous model if a retrain validates worse.

15 Conclusion

We delivered a reproducible pipeline that cleans the AirQo data without inventing PM2.5 targets, removes PM10, profiles sensors by offline time, fills gaps with three methods that were tested against hidden values, handles extreme readings, and compares thirteen models under three validation setups, with SHAP explanations, sensor maps and an external test. The best forecast model, a tuned SVR, reaches MAE 9.34 μg/m^3 on held-out data; tree models that also use the last three hours of PM2.5 reach 8.43 μg/m^3 and are far more stable. SMOTE and the bidirectional LSTM imputer were both implemented and tested: SMOTE made every model worse and was left out, and the bidirectional imputer is a better imputer but does not improve prediction. The main lesson is that, without PM10, the recent PM2.5 readings are the only strong predictor in this data, and that models trained on eight days do not transfer to a new month unless they are given those readings.

References

[1] S. Febrian and S. D. Permai. PM2.5 index prediction in Central Jakarta using pollution and meteorological data with CNN-LSTM. *E3S Web of Conferences*, 680:00067, 2025. `https://doi.org/10.1051/e3sconf/202568000067`.

[2] D. Okure, J. Ssematimba, R. Sserunjogi, N. L. Gracia, M. E. Soppelsa and E. Bainomugisha. Characterization of ambient air quality in selected urban areas in Uganda using low-cost sensing and measurement technologies. *Environmental Science & Technology*, 56(6):3324–3339, 2022. `https://doi.org/10.1021/acs.est.1c01443`.

[3] R. Sserunjogi, J. Ssematimba, D. Okure, D. Ogenrwot, P. Adong, L. Muyama, N. Nsimbe, M. Bbaale and E. Bainomugisha. Seeing the air in detail: hyperlocal air quality dataset collected from spatially distributed AirQo network. *Data in Brief*, 44:108512, 2022.

[4] I. C. Gürtepe, I. T. Senkal, A. Ünal, G. Güllü, S. Y. Aslanoğlu and J. D. Marshall. Machine learning driven regional prediction of $PM_{2.5}$ concentrations in the Eastern Mediterranean: bridging spatial data gaps in air quality monitoring. *Environmental Modelling & Software*, 192:106586, 2025. `https://doi.org/10.1016/j.envsoft.2025.106586`.

[5] J. Wang and S. Ogawa. Effects of meteorological conditions on PM2.5 concentrations in Nagasaki, Japan. *International Journal of Environmental Research and Public Health*, 12(8):9089–9101, 2015. `https://doi.org/10.3390/ijerph120809089`.

[6] Q. Yang, Q. Yuan, T. Li, H. Shen and L. Zhang. The relationships between PM2.5 and meteorological factors in China: seasonal and regional variations. *International Journal of Environmental Research and Public Health*, 14(12):1510, 2017. `https://doi.org/10.3390/ijerph14121510`.

[7] Wang et al. **[CHECK: title and full author list]**. *Environment International*, 124:371–379, 2019. **[CHECK: DOI]**

[8] World Health Organization. *WHO Global Air Quality Guidelines: Particulate Matter (PM2.5 and PM10), Ozone, Nitrogen Dioxide, Sulfur Dioxide and Carbon Monoxide*. Geneva: WHO, 2021.

[9] W. Cao, D. Wang, J. Li, H. Zhou, L. Li and Y. Li. BRITS: Bidirectional recurrent imputation for time series. *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, pages 6776–6786, 2018. arXiv:1805.10572.

[10] S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 4765–4774, 2017.

[11] S. M. Lundberg et al. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2:56–67, 2020.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[13] L. Torgo, R. P. Ribeiro, B. Pfahringer and P. Branco. SMOTE for regression. In *Progress in Artificial Intelligence (EPIA 2013)*, Lecture Notes in Computer Science 8154, pages 378–389. Springer, 2013.

[14] S. Saminathan and C. Malathy. Ensemble-based classification approach for PM2.5 concentration forecasting using meteorological data. *Frontiers in Big Data*, 6:1175259, 2023.

[15] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[16] F. Pedregosa et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[17] Evidently AI. Evidently: open-source ML and data monitoring, version 0.7.23. `https://github.com/evidentlyai/evidently`.

A Reproducing the results

File	Purpose
`train.ipynb`	The full pipeline, from raw data to deployment outputs
`requirements.txt`	Pinned package versions (Python 3.11+)
`Sensor Data Jan - July 2026.csv`	Main dataset
`AirQo_August_2026_hourly_matched.csv`	Second dataset (external test)
`data/uganda_adm1_boundaries.geojson`	Regional boundaries for the maps
`requirements-render.txt`	Packages installed by the model service on Render
`report/make_monitoring_figures.py`	Draws the monitoring figures from the Evidently outputs
`serve_model.py`	HTTP prediction endpoint (`GET /health`, `POST /predict`)
`pm25_inference.py`	Command-line batch inference from a CSV of features
`best_pm25_model.joblib`	The forecast model, target scaler and feature lists
`best_pm25_nowcast_model.joblib`	The best model with PM2.5 history
`deployment_outputs/`	August forecasts, validation comparison, metadata and run logs
`evidently_*.html`	Evidently data-drift and model-drift reports
`report/pipeline_update_notes.md`	Detailed notes behind this report

To reproduce locally: `pip install -r requirements.txt`, then run all cells of `train.ipynb`, or execute it unattended with `jupyter nbconvert -to notebook -execute train.ipynb`. The run takes about an hour on a 16-core CPU.